Dissertation Proposal: Toward Correct Natural Language Code Generation through Formal Query Languages and Formal Verification

Aaron Councilman

January 21, 2026

Contents

1 Introduction **1**

2 Related Work **4**
2.1 LLM-based Code Generation . . . 4
2.2 Program Synthesis . . . 5

3 Formal Verification of LLM-Generated Ansible **6**
3.1 Background: Ansible . . . 6
3.2 General Approach . . . 8
3.2.1 A Theory of Natural Language Code Generation . . . 8
3.2.2 An Approach to Verified Natural Language Code Generation . . . 9
3.3 Astrogator: Verified LLM-Generated Code for Ansible . . . 11
3.3.1 Formal Query Language . . . 11
3.3.2 State Calculus and Program Verifier . . . 13
3.3.3 Compiling Formal Queries to the State Calculus . . . 16
3.3.4 Compiling Ansible Programs to the State Calculus . . . 16
3.3.5 Formal Query Language Knowledge Base . . . 18
3.4 Evaluation of Natural Language Code Generation using Astrogator . . . 18
3.4.1 Benchmark Suite . . . 18
3.4.2 LLM Code Generation . . . 20
3.4.3 Verifier Accuracy . . . 20

4 Formal Verification of LLM-Generated Bash **24**

5 Automation for Formal Query Language System Development **27**
5.1 Bash Utility Specifications . . . 27
5.2 Bash Formal Query Language and Compiler . . . 29
5.3 Bash Formal Query Language Knowledge Base . . . 29

6 Conclusions **31**

Bibliography **32**

Chapter 1

Introduction

In recent years, Large Language Models (LLMs) have proven to be surprisingly successful at generating code from natural language prompts [29, 44, 56]. The apparent success of LLMs at this task has led some to speculate that prompting LLMs to generate code will fully replace traditional programming, but programming in natural language has many challenges [17] and using LLMs for this purpose has even more [65]. For example, LLMs are known to suffer from hallucinations in code generation, including generating code that uses non-existent APIs and libraries [67] and LLMs have also been shown to struggle with many practical programming tasks that rely on complex functions [47, 69] and involve reasoning about large codebases [33]. More concerningly, studies on the use of AI Coding Assistants suggest that logical bugs in LLM-generated code are often subtle [19], that many users struggle to evaluate whether LLM-generated code is correct [41], and that users using AI Coding Assistants write less secure code [50]. For mission-critical and safety-critical (together "critical") software, such as software in network stacks, distributed systems protocols, financial software, and embedded systems controllers for critical systems, the use of LLM code generation can pose serious risks as errors can have disastrous impacts. Research on AI Coding Assistants suggests that their use can benefit programmers by reducing the amount of code they need to write and helping recall syntax and API interfaces programmers use infrequently and so would otherwise need to look up [8, 41, 70]. This research, and the experience of many individuals who use AI Coding Assistants, makes apparent why the use of AI Coding Assistants is appealing including, and maybe especially, for writing critical code which is often complex and can contain subtle but important code. However, because of the significant risks that errors can pose, to use LLM-generated code in critical software we need to ensure that the generated code is correct.

An easy and apparent way to ensure LLM-generated code is correct is testing and prior works [13, 14] have shown that using provided tests or even generated tests can be used to increase the chance that the generated code is correct (since it passes some tests). While this can increase a user's confidence in the generated code, it is not a strong guarantee of correctness in any setting and is not sufficient in the context of critical software because tests can only guarantee the correctness of the program on the finite set of tested inputs, meaning they *never* completely guarantee correctness. In critical systems, a guarantee of correctness for only a subset of possible inputs is not sufficient because it means failure may still be possible. The assurance of correctness that tests offer is also highly dependent on their quality [10, 21], which includes factors like how well they cover the space of possible inputs (including invalid or unexpected inputs) and coverage of the code (which is impractical for testing LLM-generated code since the structure is not known a priori). Because of the many factors that good tests need to satisfy, developing good tests is difficult [7]. This challenge is further exacerbated in settings like system configuration tasks, which are critical software when used to configure critical systems, because identifying how to test the task can be very difficult; as an example, how do we test that the Python package `numpy` is installed? It might seem that running the Python command `import numpy` is sufficient but this will succeed if there is simply a `numpy.py` file in the current directory. Obviously, a more complicated script that imports numpy and tests its features would give a better guarantee that numpy was indeed installed, though even that could be insufficient still since again instead of installing numpy there could simply be a `numpy.py` file that contains appropriate definitions for the library components that are tested. Instead, the most reliable way to ensure that numpy is installed is to use pip (the Python package manager) to list the installed packages and verify that numpy is listed; while this is quite simple it is not obvious and it is therefore likely a user might not realized this is strongest way to test this property. For all of these reasons, tests are not a sufficient way to ensure that LLM-generated code is correct and so, at least in the context of critical software,

something more is needed.

Even for human-written code, because of the risk of errors in critical software and the insufficientcy of testing, significant efforts have been made to formally verify critical software systems [2, 35, 38]. Therefore, to enable the use of LLM-generated code in critical software, we believe it is essential to provide formal guarantees of the generated code. Providing a formal guarantee that the code generated by an LLM, from a natural language prompt, has the user's desired functionality is a very hard problem and poses many challenges. Perhaps the most fundamental is that there is no formal specification: the natural language prompt is inherently informal and may be ambiguous, vague, or incomplete. This means it is not even clear what property we should be guaranteeing and that traditional approaches, like program synthesis and verification, are not apparently applicable. Even if we had a formal specification, proving the correctness of LLM-generated code with respect to that specification is challenging since automated verification is quite difficult in general. Verification also requires formal semantics of the program and therefore its language, and most major languages lack formal semantics. LLM generated code also often makes use of libraries which need formal semantics as well; if the library is defined in the language itself and we have formal semantics of the language then no additional semantics is needed for the library but there are many cases where these libraries are written in other languages, for example Bash scripts invoke utilities (like the Linux utilities) which are not themselves Bash programs (they are often C programs) and even in Python many high-performance libraries (like numpy) call out to code in other languages; in cases like these the semantics of the library needs to be defined in some way as well as the semantics of the core language.

In prior work [15], I proposed a general solution to the first problem by introducing a high-level **formal query language** which is designed to be close to the syntax of natural language but is precisely and formally defined. The user's intent then needs to be formalized into this formal query language, as a *formal query*, which we believe can be done using LLMs. To ensure the formal query is correct, that it matches the user's intent, the user must themselves check and decide this. To enable this, the formal query language must be designed at a similar level of abstraction as the user's natural language description — using similar concepts and abstractions — so that it is readable and intuitive so the user can easily understand the formal query and ensure it matches their intent. This *user-validated formal query* then gives us a *reliable formal specification of the user's intent* we can use to verify the LLM-generated programs. To do this, we need a formal semantics of both the language being generated and of the formal query language and a program verifier that is able to decide whether the generated program satisfies the formal query. With all this, we can generate programs using an LLM and verify whether or not they satisfy the user's intent.

In my prior work, I developed Astrogator, an implementation of these ideas for the Ansible programming language, which is an IT automation language with a relatively large user-base and open source community. Astrogator includes a formal query language, designed to be high-level and intuitive to users, even those without programming expertise or knowledge of Ansible, and a program verifier that determines whether an Ansible program satisfies a given query. The verifier works by compiling both the program and the formal query into a common calculus (that we call the *State Calculus*), symbolically interpreting each of these resulting programs to determine their behaviors, and comparing these behaviors to determine whether the program behaves as the query specifies it should. In our benchmark suite of 21 Natural Language to Ansible code generation tasks, we showed that Astrogator proved the correctness of LLM-generated code in 83% of cases and precisely identified incorrect code in 92% of cases. The failures of Astrogator are all well understood, explained, and expected; they do not reflect limitations of our overall approach but rather of our implementation and testing set-up.

We chose Ansible as a first step towards the larger challenge of verifying LLM-generated code since we believe providing such guarantees may be easier for Domain-Specific Languages (DSLs) than for General-Purpose ones. Additionally, LLMs often struggle with generating programs in DSLs [27] and so verification may be especially important for DSLs as a way of identifying and rejecting incorrect code. We believe it is easier to provide formal guarantees for DSLs, compared to General Purpose Languages, because they enable us to use specialized domain knowledge which is important for constructing our formal query language and useful in constructing the verifier. In addition, many DSLs are not Turing Complete which can make program verification easier as verifying programs in Turing Complete languages is generally undecidable; this is not to say that no formal guarantees can be provided for Turing Complete languages, but it is an even more challenging problem as shown by the significant ongoing research on automated program verification for Turing Complete languages. Our particular approach, using our State Calculus and symbolic interpreter, was designed for Ansible but we believe can be used for other, similar, languages as well and we presented examples in our prior work of hand-translation of Bash and Arduino programs into the State Calculus.

While the theory and parts of the engineering of Astrogator can be reused to verify another language supporting

another language is still a large undertaking. In particular, to support a language like Bash we need to develop a formal query language for it, a compiler from this formal query language to the State Calculus, and a compiler from Bash to the State Calculus. The latter of these requires the development of formal specifications of not just Bash's core language but of all of the utilities (i.e., Linux utilities) that a program can invoke. For some context, the development of Astrogator took about a year and still has a number of limitations which would need additional engineering to address; the largest of these limitations is that we currently only support a small subset of Ansible modules (which are essentially library functions Ansible programs are built from). I expect that developing specifications for the many Linux utilities used in Bash programs will be a significant undertaking and so it is also worth studying how to reduce this effort, including whether these specifications could be automatically generated using LLMs and if so how to validate that the specifications are correct.

My dissertation, will focus on the work described above, including the theoretical backing of our approach using formal query languages, our implementation of these ideas for Ansible in Astrogator, and an implementation of these ideas for Bash. In developing our system for Bash I will also study how to automate the construction of such a system. The central claim of my work is that

Formal verification of LLM-generated code, against a Formal Query, is a *feasible* approach to ensure that programs generated from natural language prompts behave as intended, at least for DSLs like Ansible and Bash.

I see two key hypotheses that I must address in this work: that our approach to formally verify LLM-generated code indeed ensures that generated programs match the user's intent and that this approach is *feasible*, in particular that systems based on our ideas can be built with a reasonable amount of effort. Like we have done with Astrogator, the first of these can be addressed by studying how effectively our systems are able to correctly classify correct and incorrect code. Since I plan to explore techniques to help automate the construction of our formal query language and verifier for Bash, I believe the second of these hypotheses can be addressed by tracking the amount of the system that is built by hand versus the amount that can be constructed automatically, and as part of this, since I will explore using LLMs to generate certain pieces of the system, to study the correctness of the generated results (i.e., what portion of the generated results are correct and what portion required manual intervention to correct). The main contributions of my proposed dissertation are as follows:

1. A formalization of the problem of guaranteeing the correctness of code generated from a natural language description;
2. Our proposed solution to this problem, the introduction of an artificial formal query language that captures the user's intent in high-level but precise terms that can be reviewed by the user and used to formally verify a generated program;
3. Formal query languages and verifiers for the Ansible and Bash programming languages, showing that these ideas are applicable to at least a class of stateful languages like these;
4. An evaluation of these systems, to hopefully show that this approach is highly effective at distinguishing whether a generated program matches the user's intent; and
5. The development of techniques to reduce the effort needed to develop such systems for new programming languages.

In the remainder of this proposal, I provide background on the state of LLM-based code-generation, describe the prior work that will be part of my dissertation, and finally describe the remaining proposed work.

Chapter 2

Related Work

2.1 LLM-based Code Generation

Using LLMs to generate code has received significant attention in the past few years as LLMs have shown surprising capability to generate programs from natural language [18, 61]. However, LLMs are also known to produce code that contains a variety of errors including subtle bugs like handling of corner cases which users can find to be hard to diagnose and fix [19, 59]. Because of these mistakes, there has been lots of research trying to improve the quality of LLM generated code.

Improving LLM Code Generation Models There is lots of research attempting to improve LLM-based code generation by improving the LLMs, including by curating datasets for evaluation and training [40, 44], fine-tuning models for code [16, 39], novel training techniques [32, 62, 64, 66], and novel decoding techniques to prevent syntax errors [60] and even type errors [46]. While these techniques can improve the likelihood that LLM generated code is correct, they do not offer formal guarantees (outside of possibly syntactic and type correctness which are rarely sufficient to ensure the program matches the user's intent). In the context of coding assistants user inspection is often expected to eliminate incorrect code, though this is not an ideal as LLM-generated bugs can often be subtle. Even if there are breakthroughs from these kinds of research that means LLMs make far fewer mistakes, without techniques like explored in our work there is no still no guarantee that the code is correct and so the reason to trust it is just statistical.

Identifying Incorrect LLM Generated Code Some work has also tried to identify incorrect code by testing LLM generated code [14] which provides greater confidence that the code is indeed correct. Of course, one challenge with this technique is that it requires a strong test suite to provide strong assurances of correctness which is difficult; an example of how hard this is can be seen in the fact that errors were found in the ground-truth solutions to the well known HumanEval benchmark suite [42], despite this suite's tests. To address the challenge of constructing tests, LLMs have been used to generate tests [13] and these tests can be used to identify incorrect code, but using LLM generated tests means that the strength of the assurance that the code is correct is dependent on the quality of the tests generated by an LLM, and so may vary significantly and unbeknownst to the user. Other works have tried to go further than just tests and instead generate post-conditions for functions and using these as assertions during testing [22]. This showed that LLMs could generate reasonable formalizations which could be used to check generated code, but because these are not actually verified, just used in testing, they are not guaranteed to be correct. There is also no guarantee that such post-conditions are sufficiently strong (again, the strength depends on what the LLM generates) meaning this still does not provide formal guarantees that the code is correct.

LLM Generated Proofs LLM code generation has been explored for generating proofs in proof assistants such as Rocq and Isabelle [23]. In this context, there is no risk of the LLM producing an incorrect proof since the theorem prover checks the proof's correctness. While in this context the risk of LLM mistakes is able to be eliminated, it does not generalize to general code generation where oracles for verifying correctness do not exist; additionally this only works because the user provides a formal specification of the theorem statement. In order to mitigate the latter

issue, *auto-formalization* has been explored to convert natural language theorem statements into formal ones [63], but then there becomes a risk that the formal theorem statement is incorrect and does not represent what the user intends and therefore the proof is meaningless. While a user could check the formal theorem statement to ensure it is correct, these theorem statements can be quite complex and detailed; since it is hard for users to identify and fix bugs in LLM generated code there is no reason to believe checking these theorem statements would be any easier.

Summaries of LLM Generated Code Another approach that has been explored to provide confidence that LLM generated is correct is to generate a natural language summary of an LLM generated program and present this summary to the user [9, 43]. This enables the user to review the code's behavior without having to review the details of the code itself. While this can allow a user to ensure that the code roughly matches their intention, and should help them identify significant discrepancies, we believe that generating this summary from the code may cause a user to accept a program because the description seems reasonable even if it is not correct. As a concrete example, if we have a Bash program like `sudo apt install numpy` this might be summarized as "install numpy via apt" which makes the program sound correct, even though it is not because the package name for `numpy` is actually `python3-numpy` (on Debian). Because LLMs' mistakes often seem reasonable, like this example, we believe just summarizing a program does not solve the issue that a user may believe code is correct even if it has subtle mistakes.

2.2 Program Synthesis

Program Synthesis has been the historic way of generating programs from some specification, and many different specifications have been explored [28]. The one restriction with program synthesis is that the specification needs to be a *formal specification*, meaning that natural language specifications cannot be used as they lack semantics and formal meaning. The benefit of program synthesis, of course, is that it provides formal guarantees: the generated code is guaranteed to satisfy the specification, but offering this level of guarantee is often very difficult and means that synthesis often does not scale well.

In our work, we are interested in the benefits of both LLM-based code generation and Program Synthesis: taking the formal guarantees from synthesis with the seeming scalability and ability to use natural language descriptions from LLM-based code generation. It turns out that part of overall vision (as shown in Figure 3.2) is a feedback loop between a Verifier and an LLM code generator; this bears a striking resemblance to the feedback loop found in Counterexample Guided Inductive Synthesis [1], and similar techniques. Our approach also requires user input to identify whether the ways that a candidate programs handles unspecified details match the user's intent, which is similar to techniques from interactive synthesis [49] where the user may iteratively refine the specification until the generated program matches their full intent.

Chapter 3

Formal Verification of LLM-Generated Ansible

This chapter describes my prior work towards formal verification of LLM-generated code presented in Councilman et al. [15] (this work is currently under submission and available on arXiv). I first describe the Ansible programming language we targeted in this work, then the theory and overall design we envision, and finally our particular implementation *Astrogator* and its evaluation.

3.1 Background: Ansible

Ansible is an open-source system used to automate many IT tasks involved in the management of remote systems (such as servers or virtual machines) including provisioning, system configuration, and application deployment and orchestration [6, 53]. Ansible runs on one system, known as the *controller*, connects to the remote systems being managed via SSH, and sends commands from the controller to the remote systems and information and responses back. This simple architecture enables Ansible to be used for a wide variety of remote systems, so long as they have an SSH server and Python installed (though some features have additional requirements).

Ansible programs, called *playbooks*, are written in YAML [5]; an example is shown in Figure 3.1. An Ansible playbook is composed of a list of *plays* which specify configuration to perform on a set of remote systems. In our example, there are two plays, beginning on lines 2 and 13. Each play contains some configuration information, lines 2 to 5 and 13 to 16, and a list of tasks which will be performed in-order, lines 6 and 17. In this example, our configuration for each play includes a `name` field, which provides a description of what the play does (this acts like a comment with no semantic meaning); the `hosts` to execute the play on (here we use the names `workstations` and `servers`, the machine host names and/or IP addresses these names refer to would be defined elsewhere); and the user to execute the play as on the remote machine (`become: true` specifies that the play should be executed as the user specified by `become_user` or root if that is not specified). Finally, an individual *task* may contain some configuration options (such as the `name` field, a condition under which to execute the task, or a set of items on which to iteratively execute the task) and a *module* which specifies the behavior of the task. Ansible modules can be viewed as library functions, and are distributed as *collections* such as the built-in collection `ansible.builtin` or community collections such as `community.general`. Each module defines a particular set of operation we can perform in a task. For example, in the first task, lines 7 to 12, we invoke the `pip` module from `ansible.builtin` which is a module for managing python packages [4]. Each module has a different set of arguments (here `name` and `virtualenv`) with different types and meanings. In this module, the `name` argument specifies a list of packages to install and the `virtualenv` argument is optional and if specified specifies a path to a virtual environment to install the packages in. Therefore, this task will create a virtual environment at `/home/dev/venv` (if it does not exist) and install two packages into it; since this task is inside a play running on the workstations, the entire play has the effect of installing these packages into a virtual environment on all the workstations. In the other play, we have two tasks, the first (lines 18 to 20) uses the `apt` module to install Apache server and the second (lines 21 to 25) uses the `service` module to start the Apache server service. Since these tasks are in a play running on the servers, this play will install and start Apache server on the servers. The whole playbook will execute both of these plays and therefore configure both the workstations

```yaml
---
- name: Configure workstations with a python virtual environment
  hosts: workstations
  become_user: dev
  become: true
  tasks:
    - name: Create virtual environment and install packages
      ansible.builtin.pip:
        name:
          - numpy
          - pandas
        virtualenv: /home/dev/venv
- name: Install and start apache on servers
  hosts: servers
  become_user: root
  become: true
  tasks:
    - name: Install apache server
      ansible.builtin.apt:
        name: apache2
    - name: Start apache service
      ansible.builtin.service:
        name: apache2
        enabled: true
        state: started
```

Figure 3.1: An example Ansible playbook composed of two plays which are used to configure a set of workstations and servers, respectively. The first play installs two python packages into a virtual environment on the workstations and the second play installs, starts, and enables Apache server on the servers.

and servers.

While much of the research and development in LLM-based code generation has focused on well known general-purpose programming languages, Ansible has received some attention in the past few years including the development of Red Hat Ansible Lightspeed [52], a suite of AI-based tools for working with Ansible. Research using Ansible Lightspeed's VS code plugin has shown, though, that only 40% of code suggestions sent to users were kept by the user, unchanged while 14% were modified by the user and the remaining approximately 47% of suggestions were ultimately rejected by the user [57]. This suggests that, at least currently, LLM-based code generation for Ansible is unreliable enough that a user must check the code themselves to ensure it works as expected. To understand the kinds of errors that occur in LLM-based code generation of Ansible, in our prior work we described some subtle errors we observed in a number of cases. The first is incorrect package installations; for example, to install the Python package `numpy` on Red Hat Enterprise Linux (RHEL) we can install the `pip` package `numpy` or the `dnf` packages `numpy`, `python-numpy`, or `python3-numpy` but on Debian we must install the `apt` package `python3-numpy`. We have seen LLM-generated code that uses `pip` on all systems, an incorrect package name on Debian, or `apt` on all systems; in all these cases the program fails to install `numpy` in some cases. Errors like this are subtle as they require knowledge of Ansible, to understand the generated code, but also the underlying operating systems and package managers, to know the package names and package managers that can be used. Another example comes from permanently setting an environment variable; this is done by adding a line of the form `key=value` to the `/etc/environment` file. Ansible has a module specifically for adding and replacing lines in files; to replace a line you specify a regular expression to search for and if a line matches it will be replaced. In some generated code we observe a bug where it uses a regular expression like `^key` which will match and replace any line with a variable which begins with `key`, such as `keys`, and therefore this error can result in unintentionally replacing other environment variables. These kinds of errors suggest to us that validating LLM-generated Ansible code by hand can be quite difficult because it must be examined closely.

3.2 General Approach

In my work, I am interested in providing correctness guarantees for code generated from natural language descriptions. To understand this problem, we first discuss the theory, in particular what it even means for a program to be *correct* in this context and how this can be checked. Then, with this theory in place, we describe the design for a code-generation system that provides such correctness guarantees.

3.2.1 A Theory of Natural Language Code Generation

Informally, when we talk about a program being *correct* we mean that the program acts as the user intends it to, but we cannot provide any formal guarantee of this if the user's intention is only in their mind and expressed, likely imperfectly, in natural language. As a concrete example, consider a natural description like "create a file at /some/path", this is missing significant details like the contents, owner, group, permissions, and so on. These missing details are important: the user likely has a vision that includes some of these details (for instance, they may want an empty file) and the system they want to configure may specify others (such as what users exist to own it).

To formalize this problem, let our target language be $\mathcal{T}$ (Ansible in our case). The user has some *intent* they want a program in $\mathcal{T}$ to satisfy; this is likely not a single program as there are generally many ways to achieve the same thing and there may be details the user does not care about, for instance if they want to clone a git repository, if git is already installed it is not necessary to install it but it does not hurt either. Therefore, we can describe the user's intent as some set $\mathcal{U} \subset \mathcal{T}$; note importantly that we have *no way* of directly testing membership in this set (the only option is to ask the user but this risks mistakes slipping through). With this, though, we can now formalize our informal notion of correctness: a program $p \in \mathcal{T}$ is *correct* if and only if $p \in \mathcal{U}$.

The user then writes some natural language description $\mathcal{d}$ to express their intent. At this point, we still do not have any way to determine whether a program p is correct as the only indication of the user's intent is $\mathcal{d}$ which lacks formal semantics. To fix this, we need to introduce some formal specifications that allows us to test correctness. For this, suppose that we have some formal specification language $\mathcal{S}$ (in our case the Ansible Formal Query Language discussed in Section 3.3.1) and some semantics function $[\![\cdot]\!] : \mathcal{S} \to \mathcal{P}(\mathcal{T})$, which defines the set of programs that satisfy a particular specification. We also assume that given a program $p \in \mathcal{T}$ and specification $s \in \mathcal{S}$ we can decide whether $p \in [\![s]\!]$. Finally, suppose that we have some function $\mathcal{F}$ which converts a natural language description into a formal specification.

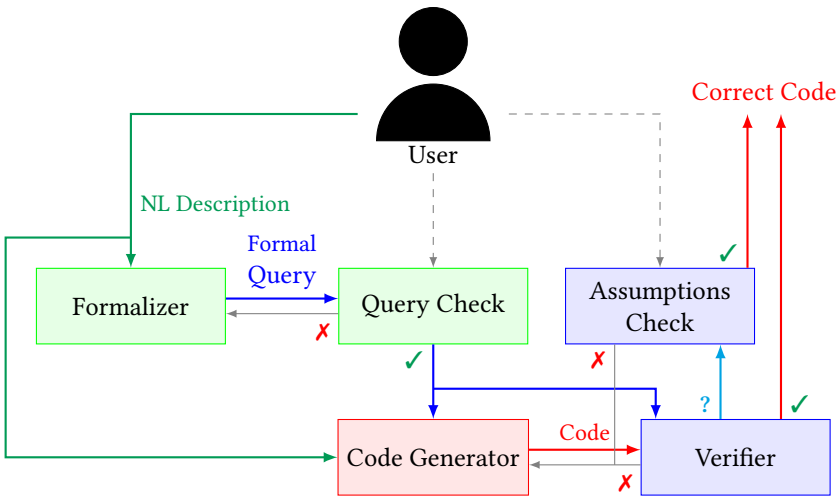

Figure 3.2: Overview diagram of our code generation system. The dashed lines between the user and Query Check and Assumptions Check indicate the user needing to review and approve or disapprove.

Because $\mathcal{F}$ takes a natural language description as input there must be instances where its output does not match the user's intent since the input may be ambiguous and $\mathcal{F}$ can only output one unambiguous formal specification. This is inherent to any attempt to automatically formalize a natural language description. The only solution to this problem is to show the generated specification $s = \mathcal{F}(\mathcal{d})$ to the user and ask them to check that it matches. Once the user approves a specification we believe that the specification s should satisfy $[\![s]\!] \supseteq \mathcal{U}$ indicating that the specification contains the user's intent but may contain other programs as the user's intent may not be fully expressed in their description and thereby the specification.

Now, we can generate candidate programs p until we find one that satisfies the specification, $p \in [\![s]\!]$. With this, we are close to a correct program but we need $p \in \mathcal{U}$, not just $p \in [\![s]\!] \supseteq \mathcal{U}$. With the information at hand: the natural language description $\mathcal{d}$ and the formal specification s, there is no way to check this. We expect that s is close to the user's intent and so if $p \notin \mathcal{U}$ this should be because of how p fills in under-specified details. Therefore, we need to identify how p differs from an *unrefined* version of s, which does not make any particular choices for the under-specified details. To do this, suppose we are able to compute an operation $p \setminus s$ which produces a high-level description of these choices. Exactly what this operation and description look like will vary depending on the target language $\mathcal{T}$, but for Ansible these differences include details like the assumption that a program is installed or a particular group exists or that the program installs an unspecified program or sets a file owner when none was specified. The result should be high-level so that it can be easily understood by the user and they can confirm whether or not the choices match their intent. By ensuring that the program $p \in [\![s]\!]$ we reduce the space of possible issues that the user must consider, but ultimately the task of determining whether $p \in \mathcal{U}$ is one that only the user can solve.

3.2.2 An Approach to Verified Natural Language Code Generation

Based on the theory described previously, we envision an end-to-end code generation system as shown in Figure 3.2 which takes a user's natural language description and generates correct code, as described above. In this figure, the "Formalizer" takes the place of the function $\mathcal{F}$, the "Query Check" represents the user needing to approve the specification s, the "Verifier" ensures that a generated program p satisfies the specification (i.e., $p \in [\![s]\!]$), and finally the "Assumptions Check" represents the user needing to approve how the program addresses under-specification of s (i.e., checking $p \setminus s$). In my prior work, we developed two key components of this system: the **Formal Query Language**, our formal specification language $\mathcal{S}$, which expresses the user's intent in a formal manner, and the **Verifier** which, given a program p and formal query s, determines whether p satisfies s ($p \in [\![s]\!]$) and is able to calculate the difference ($p \setminus s$).

As described earlier, the Formal Query Language must be understandable to the user because they must check that it matches their intention. We discuss the design considerations to satisfy this in Section 3.2.2. For our "Formalizer"

($\mathscr{F}$) we propose to use an LLM; we note that the risk of mistakes by the LLM is not an additional problem here since there is no formalizer that can be correct all the time. While we did not address the problem of generating formal queries, we believe it is suitable for LLMs, even though they are not trained on the formal query language, because in-context learning, where a few examples are provided to an LLM in the prompt, has shown efficacy at enabling LLMs to generate code in custom and domain-specific languages [34]; LLMs have also been used before to formalize natural language descriptions into formal specifications such as postconditions [22] and the generation of the formal query is not too dissimilar to the explanations of their work that LLMs often give.

Once the user is satisfied with a formal query, we can use it and their natural language description to generate programs (the "Code Generator" in Figure 3.2). Note that the code generated by this step does not need to satisfy the formal query; in my prior work we did not use the formal query in this step and simply provided the natural language description to an LLM, though we believe that providing the formal query could clarify the user's intent if there is ambiguity.

Finally, a generated program is checked against the formal specification by the "Verifier". We discuss the role of the verifier in detail in Section 3.2.2. As Figure 3.2 shows, the user may be asked to check the choices a program makes regarding the under-specification of the query ($p \setminus s$) via the "Assumptions Check". In my prior work, we could identify such assumptions (i.e., our verifier calculates $p \setminus s$) but in our evaluation we accepted all programs where $p \in [\![s]\!]$.

Design Considerations for a Formal Query Language

As described earlier, to provide any formal guarantee of correctness we need a formal specification of what the program should do. We do not need this formal specification to be complete, indeed program synthesis is often done on incomplete specifications [49], but do need this specification to be unambiguous. As a concrete example, consider the following description for a desired Ansible program: "copy a file from `/path/a` to `/path/b`." In the context of Ansible this could mean duplicating the file on the remote system, copying file `/path/a` from *the Ansible controller* to `/path/b` *on the remote system*, or from the remote system to the controller! Any of these interpretations is conceivably correct, i.e., the query is ambiguous. While we intended this example as the first interpretation, we find that all six LLMs we prompted *only* generated code to copy a file from the controller to the remote system, potentially because this is the default mode implemented in the `copy` module. This example raises a key design question:

(Q1) How do we ensure the formal specification matches the user's intent?

Section 3.2.1 provides our answer: the user must check that the specification is correct. We therefore need a language, that we call a **Formal Query Language**, that is designed to formalize the user's intent described in natural language. Since the user must check the specification in this language, which we call a **Formal Query**, we want to make it easy and intuitive for the user and so the formal query should be close in syntax to natural language so that the user can read and understand it. This means that our formal specification must be high-level and appear natural language-like. The idea of natural language-like formal languages it not new, Controlled Natural Languages (CNLs) [36], languages whose syntax is a restricted form of natural language, have been studied for many years in a variety of applications from database query languages [30] to first-order logic specifications [24, 58]. Research on the usability of CNLs has shown that such languages can be easy to understand and ensure matches the user's intent [30] which suggests that designing our formal query languages to be like CNLs can enable users to check that the specification is correct. A further design question arises, though:

(Q2) How do we ensure the user does not incorrectly believe a query matches their intent?

For example, as described previously, on Debian the package name for numpy is `python3-numpy`, but a user may not know this off the top of their head and might believe a reasonable name like `numpy` is correct. Our solution is that the formal query language must preserve certain high-level concepts, so rather than using package names we use *package descriptions* such as "numpy" or "apache server" and use a trustworthy knowledge base to determine the correct package names and similar facts. This use of high-level concepts and a knowledge base is part of why we are focusing currently on Domain-Specific Languages; the domain provides the high-level concepts that are needed and provides the information needed to construct the knowledge base. We discuss our use of these design principles to build a formal query language in Section 3.3.1.

Program Verifier

Along with the Formal Query Language, the Program Verifier is the other key component to our approach. The Verifiers's role is to take a generated program and a formal query and determine whether or not the program matches the *user's intent*. This is a significant challenge as there is no way to do it directly, we can only use the formal query and seek feedback from the user. Let us consider the earlier example of cloning a git repository to exemplify this challenge. In our experiments we find that some LLM generated programs for this will simply invoke ansible's git module (which requires git to be installed) and others will first install git before using it. Both of these are reasonable: if the user's intent is to run the program on a machine with git installed they would likely be fine with either version (though there may be circumstances where they prefer or need the former) but if the user's target does not have git installed they need the second version. A convenient solution for the Verifier would be to require that the formal query include a full description of such details, so specifying to install git if needed, but this would violate design question (Q2) as the user would need to think of these details and failing to do so, here simply failing to note whether they need to install git, could result in accepting incorrect code.

Thus, as described in Section 3.2.1, the Verifier's role is two-fold, to ensure that the program generally matches the query ($p \in [\![s]\!]$) and therefore to detect certain errors like installing the wrong package or failing to create a file, *and* the Verifier must also identify how the program fills in under-specified parts of the query ($p \setminus s$) and present them to the user who must check them. The details of how a verifier does these things will depend highly on the target language and formal query language; we discuss our approach in Section 3.3.2.

3.3 Astrogator: Verified LLM-Generated Code for Ansible

In my prior work I implemented the design described above for the Ansible programming language in a system called Astrogator. In Astrogator, we developed a formal query language and program verifier which together can be used to reliably identify correct and incorrect LLM generated Ansible code. We describe the design of our Formal Query Language in Section 3.3.1. Then we discuss the program verifier in Section 3.3.2, which is built on a novel calculus called the **State Calculus**, a symbolic interpreter for this calculus, and a unification process that matches the expected and actual behavior of a program to verify its behavior. Finally, in Sections 3.3.3 and 3.3.4, we describe how formal queries and Ansible programs, respectively, are compiled into the State Calculus.

3.3.1 Formal Query Language

As discussed already, the role of the Formal Query Language is to provide a formal expression of the user's intent and must do so in a way so as to be understood, and reviewed, by the user. We identified two key questions to consider in designing our Formal Query Language; our answers are:

(A1) The formal query language must be sufficiently high-level and interpretable that a user can easily understand whether a query matches their intent or not.

(A2) Determining whether a query matches the user's intent must not rely on specialized knowledge (such as package names or assumptions made by Ansible modules).

These principles ensure that the user can verify a given query is correct and that errors do not go undetected by the user.

From these principles we have developed a Formal Query Language for Ansible; a fragment of the grammar is shown in Figure 3.3 and a number of examples are shown in Table 3.1. Towards principle (A1), queries are designed to mimic the structure of natural language; they are sequences of sentences which are, in turn, sequences of basic atomic queries or a conditional sentence like the fourth example, where `if zsh configuration file for user="dev" not exists` allows the query to specify what should happen in different circumstances. Atomic queries specify simple imperative actions and are defined by a verb (such as `move` in the first example or `copy` in the second), a description (like the package description `zsh` or file desciption `zsh configuration file` in the fourth query), and finally additional arguments defined either by a separator and a value (such as `at /user/home/venv` in the third query) or a separator followed by key-value argument pairs (like `with glob="*.war"` in the second example). Separators act both to separate the description from the arguments and when followed by a value act as the variable

query ::= ε | *sentnc* | *sentnc* . *query*
sentnc ::= *atom* | *atom* ; *sentnc* | *atom* ; and *atom* | if *cond* then *sentnc* otherwise *sentnc*
atom ::= *verb desc args*
cond ::= *cond* or *cond* | *cond* and *cond* | os is Debian | os is RedHat | *desc args* exists | ···
verb ::= create | delete | copy | move | enable | disable | install | ···
desc ::= ε | file | bashrc file | directory | user | group | numpy | apache server | ···
args ::= *sep binds* | *sep val*
sep ::= to | from | at | with | ···
binds ::= *id* = *val*[, *binds*]
val ::= (*id* | *string* | *int*)*

Figure 3.3: Grammar of the Ansible Formal Query Language

Natural Language Description	**Formal Language Query**
Move the file at /scratch/file.txt to /home/user/data.txt	**move** file *from remote "/scratch/file.txt" to remote "/home/user/data.txt"*
Copy all .war files from /tmp on the remote machine into /backup on the controller	**copy** files *with glob="*.war" from remote /tmp to controller /backup*
Create a Python 3.10 virtual environment at /user/home/venv and install numpy in it	**create** virtual environment *with python="3.10" in /user/home/venv*; **install** numpy *in virtual environment at /user/home/venv*
Install zsh, set it as the user 'dev"s default shell, and ensure its configuration file exists	**install** zsh; **set** default shell *for user="dev" to zsh*; if zsh configuration file *for user="dev"* not exists then **create** zsh configuration file *for user="dev"*
Enable passwordless sudo for a 'wheel' group and create a 'ansible' user in that group	**enable** passwordless sudo *for group=wheel*; **create** user *with name=ansible in supplemental groups=wheel*

Table 3.1: Example Natural Language descriptions and Formal Language Queries for Ansible programming tasks. Colors and fonts correspond to the non-terminals in Figure 3.3.

name, so `to remote "/home/user/data.txt"` in the first example sets the `to` argument (specifying the destination) to the given path.

In following the (A2) principle, our design reduces the effort required by users to verify a query by preserving high-level descriptions, such as `numpy` in the third example (rather than using the actual package name), `zsh configuration file for user="dev"` in the fourth (rather than using the actual path), and `passwordless sudo` in the fifth (rather than describing the changes needed to the sudoers file). Details on how we achieve this are discussed in Section 3.3.3.

As a concrete example, consider the following Ansible program:

```
---
- name: Install and start apache on servers
  hosts: servers
  become_user: root
  become: true
  tasks:
    - name: Install apache server
      ansible.builtin.package:
        name: "{{ 'apache2' if ansible_os_family == 'Debian' else 'httpd' }}"
        state: present
    - name: Start apache service
      ansible.builtin.service:
        name: "{{ 'apache2' if ansible_os_family == 'Debian' else 'httpd' }}"
        state: started
```

This program includes a single play with two tasks: the first installs Apache Server and works on both Debian-based operating systems (where the package name is `apache2`) and on RHEL and related operating systems (where the package name is `httpd`); the second task then starts the Apache Server's service, whose name again depends on the operating system. A formal query for this problem is: `if os is Debian or os is RedHat then install apache server; and start apache server service`; this describes formally what we just described in natural language. This example illustrates that our formal queries are significantly easier to understand than the Ansible code, are close to natural language in structure while still being precise, and are easier to verify the correctness of than the playbook itself.

3.3.2 State Calculus and Program Verifier

In order to verify the correctness of Ansible programs we compare the behavior of the program and expected behavior defined by the formal query. To do this, we developed the **State Calculus** which provides a clean mechanism to describe the semantics of Ansible programs. Given an Ansible program and a formal query, both compiled into the State Calculus (see Sections 3.3.3 and 3.3.4 for details), we make use of a symbolic interpreter to identify their behaviors which we then compare to determine whether the program satisfies the query and identify any differences.

The State Calculus

Programs in languages like Ansible and Bash are executed mostly for side-effects in the form of changes to the system state. We can model such programs as executing with some current system state they read information from (such as whether a file exists) and make modification to. This gives rise to our **State Calculus**, which is a simple calculus designed to model programs that operate in this manner.

States We define states generically, so as to be applicable to Ansible, other languages, and even other domains. States, denoted by σ, are defined in Figure 3.4a; formally, they are a pair of functions, but we can think of it as a set of two kinds of values: attributes and elements. Attributes, described by *attribute labels* from some set $\mathcal{A}$, map to a value v; these attribute values are used to describe information in the state that has a value. For example, we might have an attribute os whose value describes the operating system we are running on or an attribute file_mode describing the file permissions of a certain file. Elements are used to model the existence or non-existence of certain objects in the state, such as files or users. An element is specified as an *element label* from $\mathcal{E}$ and a value v; for

$\sigma \equiv (\mathcal{A} \to v) \times (\mathcal{E} \times v \to \tau)$

$\tau ::= \sigma \mid \bullet$

$\mathcal{E}$: the set of element labels

$\mathcal{A}$: the set of attribute labels

v : the set of values

(a) Formal Definition of States

$\mathfrak{s} \equiv (\mathcal{A} \rightharpoonup v) \times (\mathcal{E} \times v \rightharpoonup \tau)$

(b) Symbolic State Definition

$p ::= s^*$

$s ::= x \leftarrow e \mid x \leftarrow \alpha(e) \mid x \leftarrow \textbf{get}\ a \mid \textbf{add}\ c \mid \textbf{ret}\ e \mid \textbf{fail}$

$\mid \textbf{if}\ e\ \textbf{then}\ p\ \textbf{else}\ p \mid \textbf{exists}\ l\ \textbf{then}\ p\ \textbf{else}\ p$

$\mid \textbf{case}\ e\ \textbf{of}\ \mathbf{L}(x) \Rightarrow p\ \mathbf{R}(x) \Rightarrow p$

$\mid \textbf{foreach}\ x\ \textbf{in}\ e\ \textbf{do}\ p$

$e ::= x \mid \ell \mid \langle e, e \rangle \mid \rho(e) \qquad a ::= \mathcal{A} \mid t.\mathcal{A} \qquad c ::= a \Leftarrow e \mid l$

$l ::= t \mid \overline{\mathcal{E}(e)} \mid t.\overline{\mathcal{E}(e)} \qquad t ::= \mathcal{E}(e) \mid t.\mathcal{E}(e)$

x : variables $\quad \alpha$: stateful functions $\quad \ell$: literals

ρ : pure functions

(c) Syntax of the State Calculus

Figure 3.4: Description of the State Calculus

instance, if file $\in \mathcal{E}$ then file(`/etc/passwd`) is an element referring to the `/etc/passwd` file. Where attribute labels identify a piece of information, an element label identifies a class of objects and when applied to a value it identifies a particular object. In our states, elements map to either the special value $\bullet$, indicating the element does not exist, or to a *nested state* σ, which allows us to apply attributes on an element. For example, the file(`/etc/passwd`) element can map to a state with attributes describing the owner, the file permissions, and so on. As an example, a state with file(`/etc/passwd`) $\mapsto$ [owner $\mapsto$ `root`, mode $\mapsto$ `0644`], file(`/root/notes.txt`) $\mapsto \bullet$ indicates that the `/etc/passwd` file exists, is owned by root, and has permissions 0644, and the file `/root/notes.txt` does not exist.

Syntax The grammar of the State Calculus is shown in Figure 3.4c, much of which is fairly standard. A program p is a sequence of statements s which are a mix of standard and unique forms. These begin with variable assignment $x \leftarrow \cdots$ where the right hand side is a pure expression e (here pure means they do not read or modify the state), or special forms $\alpha(e)$ or **get** a. $\alpha(e)$ represents the invocation of a *stateful function* α. For Ansible, these stateful functions include the Ansible modules but can be any function that can be defined by a program in the State Calculus. The other unique form, **get** a, is used to read the value of the attribute a; which is either an attribute label from $\mathcal{A}$ or an attribute label on an element $t.\mathcal{A}$, where the element t can be a single element, $\mathcal{E}(e)$, or a sequence of nested elements. For example, we load the owner of a file with $x \leftarrow$ **get** file(`/path`).owner. The next statement form, **add** c, is used to modify the state by assigning a value to an attribute, $a \Leftarrow e$, or adding an element l. As an example, the statement **add** file(`/path`).owner $\Leftarrow$ "user" sets the owner of the `/path` file. We can also specify a *negated element*, $\overline{\mathcal{E}(e)}$, to remove an element (i.e., map it to $\bullet$). So **add** $\overline{\text{file(/path)}}$ represents deleting the file. The next two forms, return and failure, are standard.

The remaining four statement forms are control flow: if-then-else statements, for-each loops, and case statements are all standard and behave as expected. The fourth is the *exists-then-else* statement which acts like an if-then-else statement but the condition tests whether an element l exists (does not map to $\bullet$). The exists-then-else statement allows us to read elements from the state and serves as a complement to **get** which reads attributes. Finally, the expression forms in the calculus are mostly standard, with variables x, literal values ℓ, pairs $\langle e, e \rangle$, and finally function calls to *pure functions* $\rho(e)$, which include built-in operators.

Symbolic Interpreter

To determine the behavior of programs in the State Calculus, we developed a symbolic interpreter that determines the behavior on all possible initial states. The result of the interpreter is a set of *symbolic initial-final state pairs* that reflect the behavior of the program under different initial states. To do this, the interpreter tracks both an initial state and the cumulative modifications made to the state (which we often imprecisely refer to as the "current state"). These initial and cumulative states are defined as symbolic states $\mathfrak{s}$, as defined in Figure 3.4b where the attribute and element maps are partial functions to record only the attributes and elements that are relevant to the program. The design and implementation of the interpreter are mostly straightforward. The main complexity is handling symbolic

values and the state which we discuss next.

The three constructs dealing with the state are **get**, **add**, and **exists**. Of these, **add** is the simplest, we simply record the addition into the current state. For **get** and **exists** there are two scenarios, either we can determine the result based on the existing state or the result depends on undefined parts of the state. The first scenario is the simplest; for a **get** we search the state for the attribute and find a value for it in the current or initial states and we simply use that value (if we find the attribute in both we use the value in the current state as this is the up-to-date value). Similarly, for **exists**, we find the element bound in the current or initial state (again preferring what we find in the current state) and take the appropriate branch.

The other scenario is that the attribute or element being read is undefined. In this case, to interpret the statement, we need to define this missing piece. For **get** all we need to do is add the attribute to the state and give it a symbolic value representing that it can have any value. The one complexity is determining whether to add it to the initial or current state; in general we add it to the initial state (since generally the properties attribute represent depend on the initial system configuration) but if the attribute is nested on an element not present in the initial state we add it to the current state instead. As examples, the first case occurs when a file existed originally and we read the owner which is some (unknown) value while the latter case occurs when a file was created and we read the owner which is some unknown value as it was not explicitly set. Handling **exists** is similar, except we have to handle both cases: where the element exists or not (is bound to $\bullet$). To do this, we will separately update the state for both cases and interpret the **then** and **else** branches respectively. For updating the state we always update the initial state since elements cannot be unspecified like attributes can and so much always originate from the initial state.

The last interesting pieces of the interpreter are how symbolic values are handled by if-then-else, case, and foreach statements. For if-then-else and case statements when the condition or scrutinee is symbolic we must interpret both branches, like we did for **exists**. When we do this we record the value assumed for the symbolic value (i.e., true or false for a condition based on the branch we take) so the value can be reused again later if needed. Handling for-each loops over symbolic values is more challenging. For a concrete list, we simply interpret the body of the loop once for each element of the list, but for a symbolic value we do not know the elements but still want to execute the loop body so that the side-effects of the loop are performed. To do this, we create a new symbolic value to represent the elements of the list and interpret the loop body *once* using this symbolic value as the element. We then record that this symbolic value represents all elements of the list, information which is used in our verifier, as described below. While this approach to handling symbolic lists is imprecise, loops in Ansible are simple enough that this approach captures their general behavior.

Program Verifier

Finally, to verify that the behaviors of the program and query match we compare the results of interpreting them; we call this component the *unifier*.

Basic Operation By compiling the formal query and Ansible program to the State Calculus we can run the symbolic interpreter to produce a set of symbolic initial-final state pairs for each (we refer to these as the query's and program's state pairs, respectively). To verify that the program's behavior matches the formal query's we need to ensure that they behave the same under the same initial conditions; the initial conditions are represented by the initial state and the behavior of the program (i.e., what it side-effects is has) by the final state. To do this, for each of the formal query's state pairs s we need to find one of the program's state pairs q such that q's initial state is *consistent* with s's and their final states are the same; such a pair indicates that under the initial conditions described in s, the program behaves the same as the query. Note that we only require q's initial state to be *consistent* with s's, rather than the same; this is because the program is not required to rely on all the assumptions that the query does. For example, a query like `if os is Debian install git` will have an initial state that assumes the OS is Debian, but the program may not ever check the OS since it is not needed (since the package name is always the same) and so that assumption may not appear in the initial states of the program. The final states we do require to be the same, since the program must behave the same as the query, and if anything is missing from the final state it means the program did not behave as expected.

Because the interpreter's results often involve symbolic values, this process also involves some unification-like techniques where symbolic values may be unified or concretized. As an example, based on a problem in our benchmark suite, suppose we have a query like `if os is Debian then create file "Debian". if os is RedHat then create file "RedHat".`, when we interpret this query these two file names appear directly in the resulting

state pairs but a generated program might just use Ansible's OS family variable as the filename because it has the appropriate value, and so in the program's resulting state pairs the specific names "Debian" and "RedHat" are not used, instead a symbolic value is used. This program works, and so to handle this we need to be able to concretize symbolic values in the program's state pairs into concrete values to match the query's. We can also need to unify symbolic values between the query's and the program's state pairs, for instance when both have a file with some unknown contents; note that we will only unify a symbolic value which represent elements of a symbolic list with another value that represents the value of another symbolic list (and that we can unify their symbolic lists), this ensures that we preserve the knowledge that they represent many values at once. Note that we cannot concretize symbolic values in the query's state pairs to match a concrete value in a program's state pair, because a symbolic value in the query's state pair indicates that the behavior is expected for all possible values and so the program should work for all values as well.

Handling under-specified queries As discussed in Section 3.2.2 it is often the case that a formal query is an under-specification and this means that a generated program may be valid even if makes additional assumptions (like that a certain program is already installed) or performs additional actions (like installing an additional program) compared to the formal query. In addition to allowing this, the unifier also needs to accumulate these assumptions and actions so they can be shown to the user (this is $p \setminus s$ as described in Section 3.2.1). To do this, we have to change how we check initial states for consistency and loosen the requirement on final states. For initial states, we already allow programs to have fewer assumptions than the query and now we need to also allow the program to have additional assumptions; thus consistency is now just that if the program's and query's initial states contain a common assumption then it is consistent (i.e., they do not disagree about the existence of an element or the value of an attribute); any assumptions in the program's initial state not present in the query's initial state are reported as additional assumption. Similarly, for final states, we had previously assumed that the program's and query's final states must be the same but now we permit the program's to include additional actions (i.e., additional elements or attributes) and we report any of these as additional actions; importantly it is still the case that all actions in the query's final state must be matched exactly in the program's final state to ensure the program performs the actions expected of it.

3.3.3 Compiling Formal Queries to the State Calculus

Compiling a formal query to the State Calculus is mostly straightforward; a query is a sequence of atomic queries, the generated code of which are composed together in sequence. Conditional queries are compiled into if-then-else statements after compiling the condition; boolean combinations of conditions are compiled to pure functions that perform these operations, and the base conditions, such as checking the OS or whether a file exists, are compiled into specific code for that purpose such as reading an attribute that represents the OS and checking its value.

Compilation of atomic queries is based on the verb and description. For example, creating a file and a directory generate different code, but creating a file always generates the same code regardless of the description of the file: we add a file element with the appropriate path and set attributes based on the arguments provided. Similarly, installing a package is always the same: we add an element indicating the package's presence and set attributes like the package manager. Other atomic queries are similarly straightforward. Currently the set of verbs we support, and the outline of how to compile them, are built-in to the compiler but we believe these can be extended easily, such as through our knowledge base which is already used for handling descriptions (see Section 3.3.5).

3.3.4 Compiling Ansible Programs to the State Calculus

To compile Ansible programs into the State Calculus we use a relatively simple compiler to translate playbooks. This compiler relies on definitions of Ansible modules (modules are like library functions that Ansible programs are built from) that provide type information and the semantics as a program in the State Calculus. To facilitate these definitions, we develop a *module definition language*. We describe the Ansible compiler and module description language in turn.

Ansible Playbook to State Calculus Compiler

The actual compilation of an Ansible program into the State Calculus is surprisingly simple. We generate code for each task in each play in sequence and so the core of the procedure is just generating code for an individual task.

```
1  module ansible.builtin.copy -> bool {
2      (dest: path)
3      (src: path | content: string)
4      if provided src {
5          [remote_src: bool]
6          let src_loc = provided remote_src
7              ? (remote_src ? file_system::remote : file_system::controller)
8              : file_system::controller;
9          if exists file(src, src_loc) {
10             file(dest, file_system::remote).content = file(src, src_loc).content;
11         } else {
12             return false;
13         }
14     } else {
15         file(dest, file_system::remote).content = content;
16     }
17     return true;
18 }
```

Figure 3.5: A simplified module description for `ansible.builtin.copy`

The module itself becomes an invocation of a stateful function for that module and other features, like conditional execution, storing the returned value, and iterating over a list are straight-forward to handle. The main complexity then is constructing the input as this relies heavily on type information to ensure the arguments used are appropriate and have the correct types. This compilation must also identify built-in Ansible variables, such as those providing information on the OS, and convert them into reading appropriate attributes.

Module Description Language

As mentioned above, the definitions of modules and their types are essential to compiling and interpreting Ansible programs. It would be ideal if this information could be extract from an existing source, such as a module's documentation or (Python) implementation but doing so, and ensuring the resulting definitions are correct, were beyond the scope of our prior work. Regardless of how we acquire these definitions, we needed some language to specify them in; we developed the module description language for the purpose of defining the behavior and types of modules. The language includes features for specifying Ansible's module argument, which include optional arguments, some with default values, and sets of arguments where exactly one is required. Many Ansible modules also have arguments that must take a value from some list of values which we represent with custom enum types. The module description language also enables defining new elements and attributes as needed for the modules being specified.

Figure 3.5 shows a simplified definition for the Ansible copy module, the omitted details deal with the many additional arguments (like file owner and mode) the module supports and details for copying directories. This demonstrates the features we have described. Arguments are defined on lines 2 (`dest` is a required argument), 3 (exactly one of `src` and `content` is required), and 5 (`remote_src` is optional). When arguments are optional, or one of several options, we can check if a particular one was provided like on line 4 where in the then-branch `src` was provided and can be used (see line 10) while in the else-branch `content` was provided and can be used (see line 15). Similarly, we check whether `remote_src` was provided on line 6. The module's behavior relies a lot on the state, for instance line 9 ensures the source file exists, and lines 10 and 15 set the contents of the destination file based on the source file and `contents` argument, respectively.

We have designed the module language to make it easy to write specifications for modules, since a specification is needed for every supported module. Towards this goal, it gives significant flexibility in deciding the level of detail tracked in the state. For example, we model file systems at a rather coarse level, like shown in Figure 3.5, by just tracking the path and system we are referring to, but this approach does not model hard-links, where two files share the same inode, correctly; if we care about such details the module language provides the flexibility to model them, perhaps by defining an inode attribute for files and having an inode element that maps an inode number to its contents. We have also found that the language makes it easy to add features of a module as needed by adding the

extra arguments and defining their uses as needed.

3.3.5 Formal Query Language Knowledge Base

As described briefly earlier, we make use of a Knowledge Base in compiling formal queries into the State Calculus. This Knowledge Base is essential in compiling high-level descriptions of packages, files, and so on from the Formal Query Language into the State Calculus. This is essential to the design of the Formal Query Language as it ensures that users do not accept incorrect but plausible sounding queries, which as we have described previously is an issue with LLM-based code generation. The knowledge base provides a set of functions that lower these high-level descriptions into lower-level ones appropriate for the State Calculus. For example, a `pkgDef` function takes in a package description and context information (including what OS is being targeted) and returns the appropriate package name and package manager; for instance, if the package description is "apache" or "apache server" and the OS being targeted is Debian-based it will return the package name `apache2` and the `apt` package manager while if the OS is RHEL it will return the name `httpd` and `dnf` package manager. Similarly, we have a `fileDef` function which given the description "zsh configuration file for user=foo" or "zshrc file for user=foo" will respond that this refers to the `.zshrc` file in foo's home directory on the remote machine.

In our implementation, we define an interface for the knowledge base which contains these various functions we need and then parameterize our compiler by an instance of this interface. This makes it easy to replace and modify the knowledge base in the future. Currently, our knowledge base has been hand curated but it can be easily extended; I propose later on to explore automating this construction.

3.4 Evaluation of Natural Language Code Generation using Astrogator

To evaluate our approach, we studied Astrogator's ability to differentiate correct and incorrect LLM-generated code. We describe below the novel benchmark suite of natural language descriptions and formal queries for common Ansible tasks (Section 3.4.1) we used, describe our code-generation methodology and the quality of the LLM generated code (Section 3.4.2), and finally discuss the system's accuracy (Section 3.4.3).

3.4.1 Benchmark Suite

For our evaluation, we needed natural language descriptions and formal language queries for tasks to be done in Ansible. Prior work on Ansible code generation used the `name` fields from tasks and plays as natural language descriptions [51] but in our experience these fields are generally quite short and often lack important details, like operating systems and paths.

Given the lack of a suitable existing benchmark suite, we developed our own by identifying common tasks based on top posts on StackOverflow and the Ansible Forum. In addition, we added tasks based on examples found in Ansible Galaxy, a large repository of Ansible code, some interesting variations on other tasks, and tasks that highlight challenges described earlier. For each benchmark we developed a natural language description, a formal language query, and a reference solution. We produced a set of 21 benchmarks which require an average of 1.67 tasks in our reference solutions and a maximum of 4 tasks, putting them in line with the size of playbooks generated by prior work [51]. The natural language descriptions and formal queries included in our benchmark suite are shown in Table 3.2. The benchmark suite includes a mix of tasks involving file, user, and package operations.

To evaluate how well Astrogator is able to distinguish correct and incorrect programs we need to know whether the LLM-generated programs we evaluate on are themselves actually correct or not. To do this we use tests because, even though they are imperfect, no other solution was feasible: as described in Section 3.4.2 we have 1260 programs, making manual inspection infeasible. We have done our best to craft strong tests that address subtle aspects of our problems. To test generated programs, we run them on a set of virtual machines along with setup and testing scripts which determine whether the program behaved as expected. We use multiple tests to test the behavior under different initial conditions, such as for problem 6 where a file should only be created if it does not already exist. We used VMs running Debian 12.11.0, Ubuntu 24.04.2, and Red Hat Enterprise Linux (RHEL) 9.6; for any problem that does not specify an operating system we test the program on each system and for problems that specify certain operating systems we run it on those systems. Between each program execution we reset the VMs from a snapshot to ensure that a program cannot impact the behavior of another. A program is judged to be correct only if it passes all tests on all systems it was tested on.

#	Natural Language Desciption	Formal Language Query
1	Create directory /srv/www	create directory at /srv/www
2	Delete the directory /home/mydata/web	delete directory at /home/mydata/web
3	Delete the contents of the /home/mydata/web directory	delete files in /home/mydata/web
4	Copy the file /scratch/file.txt to /home/user/data.txt	copy file from "/scratch/file.txt" to "/home/user/-data.txt"
5	Move the file at /scratch/file.txt to /home/user/-data.txt	move file from "/scratch/file.txt" to "/home/user/-data.txt"
6	Create the file /etc/file.txt with the contents "beginning" if it does not already exist	if file "/etc/file.txt" not exists then create file at "/etc/-file.txt" with content="beginning"
7	Copy all .war files from /backup on the controller into /tmp on the remote	copy files with glob="*.war" from controller /backup to remote /tmp
8	Copy all .war files from /tmp on the remote machine into /backup on the controller	copy files with glob="*.war" from remote /tmp to controller /backup
9	Clone the release1.8.4 branch from the ansible/ansible github repo to /ansible using https	clone github repository with name=ansible/ansible, branch="release1.8.4" into /ansible via https
10	Permanently set the environment variable LC_ALL to "C" system-wide	set environment variable LC_ALL to "C"
11	Add the line 'module load gcc' to the end of foo's bashrc	write "module load gcc" to bashrc file for user = foo at position=end
12	For Debian, restart the system if it is needed	if os is Debian and reboot required then reboot
13	Set permissions for the files and directories under /srv/path so that all users can list and read all files but only the owner can write	set file permissions in /srv/path to read=all, write=owner, list directory=all
14	Create a Python 3.10 virtual environment at /user/home/venv and install numpy in it	create virtual environment with python="3.10" in /user/home/venv; install numpy in virtual environment at /user/home/venv
15	For Debian and RedHat based systems, install numpy	if os is Debian based or os is RedHat based then install numpy
16	Install zsh, set it as the user 'dev"s default shell, and ensure its configuration file exists	install zsh; set default shell for user="dev" to zsh; if zsh configuration file for user="dev" not exists then create zsh configuration file for user="dev"
17	Disable the password for the 'service' user	disable password for user=service
18	For RedHat and Ubuntu, install the lastest version of postfix, backup its default configuration file, and download the configuration file from http://example.com/cfg.ubuntu for Ubuntu and http://example.com/cfg.redhat for RedHat	if os is RedHat or os is Ubuntu then install postfix with version=latest; copy postfix configuration file to ?backup. if os is Ubuntu then download postfix configuration file from "http://example.com/cfg.ubuntu". if os is "RedHat" then download postfix configuration file from "http://example.com/cfg.redhat"
19	For RedHat, install apache server, ensure the home page exists, and create a 'webdev' user	if os is RedHat then install apache server; create user with name=webdev; if apache server html home page file not exists then create apache server html home page file
20	Enable passwordless sudo for a 'wheel' group and create an 'ansible' user in that group	enable passwordless sudo for group=wheel; create user with name=ansible in supplemental groups=wheel
21	For Debian and RedHat, install and start an ssh client and server and generated an ssh key for 'fizbaz' at /.ssh/fizbaz_rsa	if os is Debian or os is RedHat then install ssh client; install ssh server; start ssh server service; create ssh key for user="fizbaz", name="fizbaz_rsa"

Table 3.2: Benchmark suite Natural Language Descriptions and Formal Language Queries.

Model	Error Kind			Correct
	Syntax	**Execution**	**Test Failure**	
Deepseek Coder	0	87	70	53
GPT-4o	2	65	35	108
Granite Code	0	89	66	55
Llama 3.1	22	115	40	33
Qwen2.5 Coder	4	155	20	31
Starcoder 2	14	90	52	54
Total	42	601	283	334

Table 3.3: Summary of the error types of code generated by model. Each model produced a total of 210 programs. Non-terminating programs are included as an Execution error and manually identified incorrect code is included in the Test Failure errors.

3.4.2 LLM Code Generation

To generate a diverse set of programs to test Astrogator, we selected six LLMs and, since the LLMs are nondeterministic, prompted each for ten programs for each problem to further increase the diversity of our program set, resulting in a total of 1,260 programs. The prompt used gives basic instructions and an instruction not to make use of shell commands, such as the `shell` and `command` modules which allow the execution of arbitrary shell code, because we do not currently support these modules and their use is unnecessary in our benchmarks and therefore discouraged [54]. Despite this instruction there were 130 instances of generated code containing them; these programs are identified as incorrect by the verifier even though in some cases they do pass the tests.

In our experiments we used five open source-models which perform well on other code-generation benchmarks: Deepseek Coder 6.7b [29], Granite 8b Code [45], Qwen2.5 Coder 3b [31], Llama 3.1 8b [25], and Starcoder 2 15b [44]. We also used a state-of-the-art closed-source model GPT-4o [48]. Because we observed the models do not always generate full playbooks (instead sometimes just generating a list of tasks) we performed post-processing to identify and extracts key elements and insert them into a template. In this process, 42 (3.3%) of the generated programs are unable to be processed due to syntactic issues. We also identified 5 programs (0.4%), all solutions to problem 21, which do not terminate because they invoke a shell command which requires user input. Of the remaining programs, we found that 596 (47.3%) fail to execute and 226 (18.0%) executed but fail the tests. In addition, the solutions to problem 18 cannot be executed (they requir accessing a URL that does not exist) but through manual inspection we identified only 1 of the 58 syntactically valid responses is correct. This means that in total, of the 1260 generated programs only 334 (26.5%) were correct.

Because of this surprisingly low success rate, we break down the performance of each model in Table 3.3. GPT-4o performs significantly better than the other models, generating 108 correct programs, or 51.4%, in contrast to only 21.5% from the open source models. This difference is most likely because the open source models are much smaller than GPT-4o. Within the open source models, Deepseek, Granite, and Starcoder all achieve slightly over 25% while Llama and Qwen only achieve around 15%. Qwen is the smallest model with only 3 billion parameters and Llama is the only non-code model included in our evaluation, which explains their low performance and may particularly explain Llama's significantly higher than average rate of syntax errors.

3.4.3 Verifier Accuracy

To determine how well Astrogator is able to differentiate correct from incorrect code, we ran it on each on the programs generated by the LLMs using our hand-written formal queries. A perfect result would be that it identifies exactly the 334 correct programs as correct and all other programs as incorrect but some programs that are correct are rejected and some programs that are not correct are accepted. Our overall results, by benchmark program, are shown in Table 3.4.

Of the 334 correct programs Astrogator accepts 277 of them (82.9%) and of the 926 incorrect programs Astrogator rejects 856 of them (92.4%). In total, Astrogator took about 70 seconds to process the 1260 programs. In this process it detects many kinds of errors, including all of the errors we have used as examples throughout this paper, as well as many errors involving using incorrect modules or module names, providing invalid arguments or missing necessary ones, providing invalid argument values, and so on. For the remainder of this section, we discuss the cases where

#	Correct Programs		Incorrect Programs	
	Accepted	**Rejected**	**Accepted**	**Rejected**
1	55	0	5	0
2	56	0	0	4
3	5	2	0	53
4	0	0	49	11
5	5	4	0	51
6	11	0	4	45
7	3	0	0	57
8	2	0	0	58
9	37	7	0	16
10	5	22	0	33
11	42	0	2	16
12	11	0	3	46
13	0	0	0	60
14	0	6	0	54
15	0	0	0	60
16	14	8	0	38
17	24	0	2	34
18	0	1	2	57
19	7	0	3	50
20	0	7	0	53
21	0	0	0	60
Total	277	57	70	856

Table 3.4: Evaluation Results for the Benchmark Suite. For each benchmark we have 60 samples and a total of 1260 programs across the full suite.

Astrogator is incorrect and how these issues can be addressed.

Of the 57 correct programs that Astrogator rejects, 18 are because they rely on features we do not support yet; 9 of these rely on features like `block` constructs and handlers, which are easy to support, while the other 9 rely on shell commands which will be more difficult to support, as it requires a specification of the behavior of shell programs, I believe this is possible and propose to support Bash, though support for scripts would require additional work as well. The other 39 programs come from problems: 9, 10, 16, and 20; these all share a common issue: the knowledge base only produces one response and these programs used a correct value (or approach) that did not match the one the knowledge base provided. For example, in problem 9 we are cloning a git repository and the programs use the address `https://github.com/ansible/ansible` while the knowledge base produced this same address except with `.git` at the end. Similarly, problem 10, which sets an environment variable, expects that we insert the line `LC_ALL=C` while the line `LC_ALL="C"` works as well. Problem 16's issue is the same, the knowledge base provides the path `/bin/zsh` for zsh but the path `/usr/bin/zsh` work as well and similarly for problem 20 where we enable sudo for a group, the knowledge base provides a particular line to be added to the `/etc/sudoers` file but multiple acceptable lines exist and it can be added to other files as well. The simple solution to these is to allow the knowledge base to return multiple values, which would fix these issues, but unfortunately the implementation is not quite as trivial; while we believe it can be resolved it is likely to be rather technical and may require changes to our handling of symbolic values. Thus, all of these incorrect rejections are likely fixable, though require varying levels of effort to fix.

Of the 70 incorrect programs that Astrogator accepts we find that 68 are the result of the queries being under-specified and are a result of the fact that we are accepting all programs that satisfy the query (i.e., without a human verifying $p \setminus s$). Of the other two programs, one is the result of an unhandled feature, Shell plugins, which we simply ignore in compiling the Ansible to the State Calculus but causes Ansible to fail to execute it; handling Shell plugins in our compilation of Ansible would immediately address this issue. The other program is due to an undocumented feature in the lineinfile module [3], this issue can be easily fixed by adjusting the module's definition given in the module description language[1]. Of the other 68 programs, 7 perform undesired additional actions that caused our tests to fail while the remaining 61 would work under certain assumptions not true in our testing environment. These issues are entirely expected given that formal queries are generally under-specified and, as discussion in Section 3.2.1, the way to resolve them is by asking a user for input. The 7 aforementioned programs that fail due to performing undesired actions are for problems 6 (creating a file if it does not exist) and 12 (rebooting the system if needed), which both involve conditionals like "if *cond* then do foo". Intuitively it seems such a query should also mean "... otherwise do not do foo" but since programs must be allowed to perform additional actions this is not the semantics of the formal query language and so these 7 programs are accepted despite always performing the action while ignoring the condition; our verifier identifies this and a user could easily choose to reject these programs based on the additional actions if they wanted.

The other 61 incorrect programs accepted by Astrogator work under certain additional assumptions, which again is expected. As an example, the five incorrect programs for problem 1 fail because they set the owner of the newly created directory to a user that did not exist. The output of the unifier in these cases reports that they assume the user exists and they own the new directory, and a user could easily identify whether this matches their intent. There is a similar issue with problem 17 where the two incorrect programs work if the "service" user does not already exist which a user can easily check. The remaining issues involve the creation of files with certain contents, for example in problem 4 we are meant to copy a file on the remote machine while the generated code copies the file from the controller to the remote which is not rejected because the target files are ultimately created but it requires the alternate file sources to have certain contents; again these assumptions are identified by the unifier and a user could easily review them.

As discussed above, while Astrogator is not always correct on whether a program is correct or not, its incorrect rejections are all addressable by adding support for additional features and the incorrect accepts are all expected as they are the result of the under-specified nature of the formal queries, the solution for them is to include a user to check whether the additional actions or assumptions they require match their intent. These results prove that the techniques and ideas we developed do work and the failures are just due to limitations in the implementation.

[1] We did not fix this simply because it is not clear if we should model the documented behavior (i.e., the actual behavior is a bug), the actual behavior (i.e., the behavior is intentional), or simply reject any program that triggers this behavior (i.e., this is undefined behavior and should be avoided)

Proposed Work

Chapter 4

Formal Verification of LLM-Generated Bash

For my dissertation, I first propose to extend my prior work to support Bash, a widely used scripting language for systems tasks. To do this, I believe that the State Calculus and its interpreter and verifier can be reused for comparing the behaviors of Bash programs and formal queries, which will help reduce the effort needed. To see why I believe this, Figure 4.1 shows an example of using the State Calculus for Bash, including (a) a Bash Script, (c) a hand-translation of that script into the State Calculus, and (b) a hand-written specification in the State Calculus that could be derived from a formal query language for Bash. Using these, and hand-written specifications of the Linux utilities used in the script, I have been able to use the State Calculus's interpreter and verifier to verify that the hand-translation of the script matches the hand-written specification. This example demonstrates a number of Bash features which are quite distinct from Ansible, including redirects, pipes, and file descriptors and shows that they can be supported by the State Calculus already. One feature that I anticipate may be challenging to support is arbitrary loops; as described earlier our handling of loops over symbolic values is pretty simple and does not support arbitrary while-loops or more complex forms of loops found in Bash. I believe that even without this support, a significant and important subset of Bash can be supported. Expanding to support arbitrary loops may prove difficult, as arbitrary loops are a common challenge in all verification work. It may be possible to partially support such loops by bounding their iterations or some similar technique; as we study further, and in particular identify the kinds of arbitrary loops present in the benchmarks we select, I will explore how these loops could be supported.

Based on the components of Astrogator, I identify four major components that will be needed to support Bash:

1) A compiler from Bash scripts into the State Calculus.

2) Specifications of the behaviors of Bash utilities (similar to those in Astrogator developed for Ansible modules)

```
tee >(wc -l >&2) < bigfile | gzip > bigfile.gz
```

(a) Example Bash script

```
let contents = fs('bigfile').contents;

env().stderr = ((env().stderr
    ^ string_of_int(count_lines(contents))
    ^ " ") ^ "\n";
fs('bigfile.gz').contents = gzipped(contents);
```

(b) Hand-written specification

```
let pipe = new_pipe();
let file = tmp_file();
fd(0).kind = fd::file('bigfile');
fd(1).kind = fd::pipe(pipe);
tee {{ files: [file] }};
fd(0).kind = fd::file(file);
fd(1).kind = fd(2).kind;
wc {{ lines: true }};
fd(0).kind = fd::pipe(pipe);
fd(1).kind = fd::file('bigfile.gz');
gzip {{ }};
```

(c) Hand-translation of Bash script

Figure 4.1: Example translation of a Bash script to the State Calculus. The script prints the number of lines in `bigfile` to stderr and gzips it to `bigfile.gz`.

used by the Bash compiler.

3) A Formal Query Language for Bash, in particular a parser and compiler into the State Calculus.

4) The knowledge base used by the Formal Query Language compiler.

I discuss each of these, focusing on what work they will require and any challenges I anticipate.

Bash to State Calculus Compiler Bash is a relatively simple language and so I expect the compiler from Bash to the State Calculus to be relatively straightforward and simple. In particular, as Figure 4.1 shows, the translation from the Bash script to the State Calculus program is quite straight-forward; like with Astrogator, this means that the main challenge in defining the semantics of Bash is in developing the specifications of the Bash utilities, which I discuss later.

I note that there is some prior work formalizing the semantics of the POSIX shell [26], which Bash is meant to be compatible with, and while I could explore using those semantics to automatically derive the compiler from Bash to the State Calculus, the work to take those semantics and produce a compiler is likely to be as much work (if not significantly more) than the actual compiler from Bash to the State Calculus; if the formal semantics were in some common representation that could be used for many languages, like the K semantic framework [55], then this work could be worthwhile, but in this case I believe that simply developing this compiler by hand is most effective. I also note that these prior attempts are formalizing the behavior of the shell itself, they include a very limited number of utilities, meaning that the significant work of formalizing the utilities would still be necessary.

Bash Utility Specifications As mentioned above, formal specifications of the behaviors of the Bash utilities are necessary to compile Bash programs into the State Calculus for verification. Like we did for Ansible modules in Astrogator, I believe that these behaviors can be specified in a new *utility specification* language, like the module description language we already developed, which makes it easy to define how a utility modifies the program state. This utility specification language will be quite similar to the module description language though will require replacing its features for defining module arguments (which in Ansible are key-value pairs) with features for defining the arguments of Bash utilities (which are a list of values using flags); these changes will be relatively simple.

A related challenge is determining the best way to represent the state of Bash programs, as these decisions can impact many utilities and even the compiler from Bash to the State Calculus. As an example, how files and file-descriptors are represented impacts any utility that interacts with the file-system or file-descriptors in any way (which is most utilities). In particular, in the hand-written example from Figure 4.1, I model file descriptors using an `fd` element that we set and reset, but properly resetting them is not generally as trivial and may not be possible as a utility could modify arbitrary file descriptors. A better representation would be, like happens when the Bash script is executed, to track file descriptors separately for each distinct "process". In practice the State Calculus does not have separate processes but it may be necessary to add some form of localized state so that when a utility modifies its file-descriptors and other local state this does not impact the rest of the script. Implementing such a feature in the State Calculus would likely be somewhat technical but I do not believe will be too difficult to implement.

The biggest challenge, then, is the development of the utility specifications for a *large* number of utilities to support a broad set of Bash programs. Because we want to support a broad set of Bash utilities, constructing these specifications would require substantial effort and so it is worth exploring how to make this process easier so that the burden of developing a system like this is reduced. I discuss this problem and my proposed approach in the next chapter.

Bash Formal Query Language The formal query language for Bash should, like the Bash compiler itself, be relatively simple and easy to develop. The grammar of the language itself should be quite simple as it just defines the structure of queries; in our formal query language for Ansible this means it defines the verbs and the general structure of *verb description arguments* but that is all; ultimately descriptions, parameter names, and argument values are allowed to be pretty much arbitrary as their meanings are identified and handled by the knowledge base. The translation from the formal query language to the State Calculus is similarly quite simple, it pulls information from the knowledge base and then incorporates that into a complete program; for example, a query like `create bashrc file for user=aaron with content="export FOO=true"` has the verb *create* ... *file* and so the description `bashrc` and argument `user=aaron` are passed to the knowledge base which returns the path `/home/aaron/.bashrc` for the file and then the compiler takes this path and generates code that creates that file and sets its content to the

specified value. This is the general design of the compiler: pull information from the knowledge base and insert the information into a generic template (such as creating a file, which is the same process and just depends on the path).

The big challenge I anticipate in this development is identifying the features needed in the formal query language to ensure that it can express a large variety of user intents and therefore programs. In developing Astrogator, I addressed this challenge by developing our benchmark suite ahead of time and then using it to guide the development of the formal query language, to ensure that all of the benchmarks were supported. I expect to take a similar approach for Bash, but since we hope to use a much larger benchmark suite that utilizes a wide range of Bash utilities, I believe that an iterative approach to developing the formal query language may be appropriate, where I start with a small set of features and then identify additional features needed by the benchmarks. Because this set of benchmarks will be large, exploring automating this process of identifying missing features is of interest as it could significantly aid in this development; I discuss ideas for this in the next chapter.

Bash Formal Query Language Knowledge Base The final piece of the system is the formal query language's knowledge base, which is used in its compilation to the State Calculus. The structure of the knowledge base (i.e., what information it needs to contain) will be determined by the Formal Query Language's compiler and so the remaining challenge is just collecting the information. Like with the Bash utilities, the challenge here is simply one of scale, to provide file and package descriptions of all the files and packages users may want to refer to will be a huge amount of information, and collecting it by hand is likely impractical and so automation of this process is necessary, which I discuss in the next chapter.

Chapter 5

Automation for Formal Query Language System Development

As described in the last Chapter, building a formal query language and verifier for Bash is a large undertaking as it requires specifications of a large number of Bash utilities and a large amount of information in the knowledge base. To make this work practical, and to make development of such a system easier for other languages in the future, I intend to explore, though our development for Bash, techniques to automate the development of a system like Astrogator. For context, developing Astrogator took me about a year probably about half of that time dedicated just to its development and there is significant further work needed to extend Astrogator to support the many more modules in Ansible. While much of the development time for Astrogator was focused on developing the State Calculus, its interpreter, and its verifier, work which does not need to be done for Bash, a significant amount of time was still consumed in developing the module specifications and much more would be needed for a more complete set.

As described earlier, I believe that automation will be particularly necessary in three places: developing the utility specifications, helping ensure the breadth of the formal query language, and filling the knowledge base. I discuss each of these in turn, including what I would like to automate and how I intend to do so.

5.1 Bash Utility Specifications

As described already, the main challenge in developing the utility specification is simply the scale. If there were existing formal semantics for Bash utilities we could potentially automatically convert them into the utility specification language to provide us correct specifications automatically but as far as I am aware there are no such semantics for many utilities. Instead, then, the best source for this information likely comes from the descriptions of utilities provided in the Linux manual pages, which generally provide detailed descriptions of the arguments and behaviors of these utilities. Given that we are hoping to extract a formal semantics from natural language the apparent approach is to make use of LLMs. This leads, though, to two further questions: 1) can LLMs generate appropriate specifications in our utility description language and 2) how do we ensure the generated specifications are correct? I believe the answer to the first of these questions is yes for a few reasons: first, the module description language in Astrogator is an imperative language that bears a lot of similarities to other languages that LLMs have been trained on, the details of how we represent the state are unique but LLMs have shown remarkable flexibility and since they have been used in other contexts to generate formal specifications [22, 63] I do not believe this should be too much of a challenge and likely comes down mostly to prompting the models appropriately. The bigger challenge is the question of how to ensure the specifications are correct. It is worth reiterating that the correctness of these specifications is vital: if they are incorrect then any guarantee we provide about a generated program may be wrong since these specification are used in determining the semantics of the Bash script.

The whole goal of this work is to provide guarantees that LLM generated code is correct and so it would be nice to be able to apply a similar approach to generating the utility specifications as we proposed above. Unfortunately it is not apparent that there is any way to do this. As described earlier, to be able to guarantee the correctness of a program we need we a formal specification we can verify the program against; formal queries are our solution to

this problem but what a formal query language for describing the behavior of Linux utilities would be is not clear and it would likely have to be so detailed that essentially the user would have to review the generated specification; ultimately I do not see a solution for guaranteeing correctness beyond user validation. The question then becomes whether we can automatically identify incorrect specifications so that we can at least gain some confidence in the correctness of the specification even before the user validation. The first apparent approach is to leverage existing tests; the GNU core utilities repository[1] contains tests for many of the utilities it defines, including Linux utilities like `cat`, `chmod`, `cp`, `mv`, and so on. We could interpret these tests, using the generated specification, to check that these utilities behalf correctly; unfortunately there are two issues with this idea: limited tests and a bootstrapping problem. First, not all of the GNU core utilities have tests and the tests that do exist are often limited, for example `echo` is a core utility but has no tests in this repository and `cat` has only four tests one testing the behavior of the `-E` flag, one testing that cat "outputs processed data immediately", one which tests the `-E` flag when used on a file under `/proc`, and one which tests the behavior when cat is reading from and outputting to the same file; according to the manual page there are nine other flags for `cat` that are not even tested. The other problem is that most of these tests rely on other utilities, for instance the tests for `cat` also use `echo`, `printf`, `sed`, and `tr` (among other utilities) and in turn some of the tests for `printf` use `cat`. Because of this there is a bootstrapping problem: if a test for `cat` fails is that because the specification for `cat` is incorrect or because one of the other utilities is incorrect? Even worse, if a test for `cat` passes is it because the specification of `cat` is correct or could the specification of another utility have an error that cancels out an error in the specification of `cat`? This issue is exacerbated by utilities like `echo` and `sed` which do not have their own tests and so the only way to ensure they are correct is to assume that if other tests that use them pass then they must be. This is again not sufficient, the `echo` utility has two really useful flags: `-n` to disable printing the last newline and `-e` to interpreter backslash escapes, neither of which are be used anywhere in the core utilities' test suite.

I propose, instead, that we can use the generated specifications to generate test inputs for utilities, use the State Calculus's interpreter to identify the expected behavior, and then from this generate Bash scripts we can actually run to check whether the utility behaves as described by the generated specification. For example, supposed we have the following simplified specification for the `mv` utility (assuming the arguments are provided as a list of strings `args` where the first argument is at index 0):

```
let src = args[0];
let dst = args[1];
file(dst).contents = file(src).contents;
clear file(src);
return 0;
```

which copies the file contents from the source file (the first path name) to the destination (the second path name). Obviously the actually `mv` utility is more complex than this, but even just considering its most basic behavior this specification is incorrect since if the source and destination specified are the same this ends up deleting the file entirely and returns zero while the actual utility reports an error and returns a non-zero exit code. If we interpret this definition with an arbitrary but equal path `p` for both arguments, the interpreter tells us that that file must exist initially and then is deleted and the utility returns with exist code 0; taking this interpretation we could produce a test script that sets up the environment (i.e., creates the initial file) then executes the expected `mv` command, and then attempts to verify the file does not exist; this looks something like

```
touch p # create the initial file (required by the initial state)
if mv p p; then # if mv returns 0
    if [ ! -f p ]; then # if file p does not exist
        exit 0 # behavior matches
    else
        exit 2 # report failure, did not behave the same
    fi
else
    exit 1 # report failure, incorrect return value
fi
```

and if we run this script as expected it will return with value 1 since the `mv` command fails in this instance.

[1] https://gitweb.git.savannah.gnu.org/gitweb/?p=coreutils.git

I believe there are several challenges to generating tests to validate the behavior of the formal specification:

1) Determining how to generate test inputs to the utility being tested.
2) Determining how to setup the system, based on the initial state, and validate the result, based on the final state.
3) Identifying incomplete specifications (such as a missing flag) a generating tests from the specification will never produce tests that use missing options.

Towards the first of these, there is significant literature about fuzzing and random testing [11, 12, 20, 37, 68] that can provide a basis to build similar techniques for our application and so I believe that this work will mostly require some study of the literature and implementation of the techniques described therein. I believe the second point will simply require some engineering: for each element and attribute in the state we need to develop a snippet of code that sets it up and tests it and then these can be combined together to create the test scripts like suggested above. This will certainly pose some challenges, for example as described earlier testing whether a package is installed is not obvious but since this is only a one time investment it is worth taking time to ensure that the generated tests are high quality. Finally, for the issue of incomplete specifications I believe we can leverage the existing manual pages, since usage information and flags are recorded in a well structured manner and so this information may be able to be automatically extracted and checked against the specification. This ensures that we do not omit any documented arguments; while it does not validate that they have the correct semantics the automated test generation approach should handle that since it will generate tests using those flags. Note that it is important that we generate both tests that run successfully and those that produce errors so that we can ensure that the assumptions the specification requires are indeed correct.

With the approach described above, I want to generate and validate specifications for many Linux utilities, but I also intend to conduct a study to show the efficacy and importance of each piece of this approach and to show the kinds of errors caught in the process. I expect that the LLM generated specifications will contain many errors, like in my prior work where most of the errors described above were based on real errors we saw in LLM generated code, but if there are kinds of errors that we do not observe in practice I may also construct intentionally flawed specifications to demonstrate that we are able to detect them.

5.2 Bash Formal Query Language and Compiler

As described earlier, the development of the Formal Query and compiler themselves are relatively simple, but there is space for automation in helping design the language. In particular, I found for Ansible that to develop the formal query language I needed a set of benchmarks that I could reference and ensure they were all supported. Since we want to support a large number of Bash utilities, and therefore a large number of different programs, our benchmark suite will be large and so it is of interest to me to explore how automation can help guide the development process. For this iterative development, it would be useful to be able to easily identify examples that are not yet supported by the formal query language and to do this I propose to use LLMs to translate natural language descriptions from our chosen benchmark suite into formal queries. To then identify examples that may not be supported, I believe it may be useful to develop some manner of sorting or filtering the examples to easily identify those that need attention. One possibility would be to sort based on the edit distance between the natural language description and formal query as a high edit distance means that they are not very similar (which is on its own potentially a problem since we want the queries to be similar to the natural language) and may suggest that the query is not a proper formalization of the problem. I would also note that being able to automatically formalize the natural language descriptions is also important for practical use of formal query languages, as we do not expect users to write them, and is also useful in the development of our knowledge base, as a large collection of formal queries allows us to identify what information the knowledge base needs to provide.

5.3 Bash Formal Query Language Knowledge Base

Finally, like with the utility specifications, the problem for the knowledge base is again scale. For example, our Ansible knowledge base provides information on packages, given a package description it produces the package

name and package manager needed and we expect to need something like this for Bash as well. To do this we need to map arbitrary package descriptions to the appropriate package name and package manager which is a huge amount of information and not really feasible to construct by hand. The biggest challenge with constructing the knowledge base is that the descriptions that the knowledge base takes as input are meant to be natural language descriptions, so for instance "numpy" as opposed to the package name `python3-numpy` or "apache server" instead of `apache2`. This makes validating the information generated very difficult and means that knowing what descriptions to even provide definitions for is non-trivial (how do we know that we want to define the phrase "apache server"?). On the latter of these, it is tempting to thinking that we could extract a list of these descriptions from existing knowledge sources, say the Debian package reference pages but the Debian reference pages for numpy[2] only contains the phrase "numpy" in its natural language description and the one for apache server[3] does not contain that phrase, instead it is always referred to as the "apache HTTP server". Even worse, for something like files where we might want to include things like `bashrc file for user=aaron`, `sudoers file`, and `apache server html home page`; to my knowledge there is no definitive list of such files, their paths, and their descriptions. The solution to identifying what information the knowledge base needs is, as alluded to above, to use the automatically formalized queries as we can run them through the compiler to extract a list of all of the descriptions that the knowledge base needs to define.

To then fill the knowledge base, I would propose then to use an LLM to generate the knowledge base response for each of the descriptions we identify. Validating these responses is difficult because they provide the formal semantics of natural language-like phrases; therefore, I propose a mix of validation approaches to hopefully increase confidence in the generated result. First, we can attempt to validate (at least in some cases) that the responses are reasonable, for example with package names we can automatically search distribution-provided lists of packages to ensure the packages exist and could even search the page for each package to ensure that the package description the package was associated with contains the package description (or elements of it) as we would generally expect that to be the case. Another validation technique we can use is testing; taking our benchmark suite (which I assume contains both natural language description and accompanying solutions) we can take the solutions and ensure that they are accepted by our verifier. Assuming that the formal query is correct then if the reference solution fails to match the query this should be due to an error in the knowledge base (since all of the other components in the validator are either manually constructed and trusted or were validated through other methods described earlier). This testing, unfortunately, only validates the completeness of the knowledge base, in particular we are not checking the soundness. For example, suppose that for the package description "numpy" our knowledge base replies that on Debian we can install either of the the apt packages `python3-numpy` or `numpy` (the former being the correct one); if our benchmark suite only contains correct programs (in this case ones that install `python3-numpy`) we will not detect that the knowledge base is providing incorrect information in this case. Addressing the soundness aspect is quite difficult. An apparent solution in this example might be to take any script that installs the package `python3-numpy` and modify it to now install `numpy` and then to verify (either by testing the scripts or using the symbolic interpreter for the State Calculus) that the resulting script behaves the same, but a change like this might not always produce the exact same behavior, instead we just expect its high-level behavior to be the same. As a concrete example, considering granting a user sudo access; this can be done either by adding an appropriate line to the `/etc/sudoers` file, or (on modern Linux) adding such a line to a file in the `/etc/sudoers.d/` directory; obviously modifying a script which does the first into one which does the second will produce a program that, at a low-level, behaves very differently despite it having the same high-level behavior. Since there is no apparent way to test these high-level behaviors, I leave the question of whether we could perform this validation open, if we identify a way to then I may include it in my dissertation, but if we cannot identify an approach this will remain as future work.

[2] `https://packages.debian.org/trixie/python3-numpy`
[3] `https://packages.debian.org/trixie/apache2`

Chapter 6

Conclusions

I propose that my dissertation present our approach to ensuring the correctness of LLM-generated code from natural language prompts. In my prior work I have developed a technique for this that introduces a *formal query language* which provides a formal specification based on the user's natural language description. We have demonstrated the potential of this approach for Ansible, in our Astrogator system, which accepts 83% of correct code and rejects 92% of incorrect code, and the failure cases in both directions are well understood and are due to implementation details or are expected due to the under-specified nature of the formal queries used.

I propose to extend this work to build a formal query language and verifier for Bash and to explore how to automate the construction of such a system. In particular, I intend to study automating the construction of specifications for Linux utilities and the formal query language's knowledge base, as well as the generation of formal queries from natural language descriptions; throughout this, I intend to explore not just how to automatically generate these but to identify how to validate that the results are correct. This proposed work would significantly reduce the amount of effort needed to implement a system like Astrogator and therefore make the development of such systems, which can ensure the correctness of LLM-generated code, easier, especially in circumstances where, like with Bash, the State Calculus and its accompanying machinery can be reused for verification.

Bibliography

[1] Alessandro Abate, Cristina David, Pascal Kesseli, Daniel Kroening, and Elizabeth Polgreen. 2018. Counterexample Guided Inductive Synthesis Modulo Theories. In *Computer Aided Verification*. Hana Chockler and Georg Weissenbacher, editors. Springer International Publishing, Cham, 270–288. ISBN: 978-3-319-96145-3. doi:10.1007/978-3-319-96145-3_15.

[2] Mouhamad Almakhour, Layth Sliman, Abed Ellatif Samhat, and Abdelhamid Mellouk. 2020. Verification of smart contracts: A survey. *Pervasive and Mobile Computing*, 67, 101227. doi:https://doi.org/10.1016/j.pmcj.2020.101227.

[3] Ansible Project Contributors. 2025. ansible.builtin.lineinfile module – Manage lines in text files. (July 2025). https://docs.ansible.com/ansible/latest/collections/ansible/builtin/lineinfile_module.html.

[4] Ansible Project Contributors. 2025. ansible.builtin.pip module – Manages Python library dependencies. (June 2025). https://docs.ansible.com/ansible/latest/collections/ansible/builtin/pip_module.html.

[5] Ansible Project Contributors. 2025. Creating a playbook. (June 2025). https://docs.ansible.com/ansible/latest/getting_started/get_started_playbook.html.

[6] Ansible Project Contributors. 2025. Getting started with Ansible. (June 2025). https://docs.ansible.com/ansible/latest/getting_started/index.html.

[7] Tom Badgett, Glenford J. Myers, and Corey Sandler. 2011. *The Art of Software Testing, 3rd Edition*. eng. Wiley. ISBN: 9781118133156.

[8] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.*, 7, OOPSLA1, Article 78, (Apr. 2023), 27 pages. doi:10.1145/3586030.

[9] Celeste Barnaby, Qiaochu Chen, Chenglong Wang, and Isil Dillig. 2024. PhotoScout: Synthesis-Powered Multi-Modal Image Search. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (CHI '24) Article 896. Association for Computing Machinery, Honolulu, HI, USA, 15 pages. ISBN: 9798400703300. doi:10.1145/3613904.3642319.

[10] Samera Obaid Barraood, Haslina Mohd, and Fauziah Baharom. 2021. Test case quality factors. *Turkish Journal of Computer and Mathematics Education*, 12, 3, 1683–1694.

[11] Lukas Bulwahn. 2012. The New Quickcheck for Isabelle. In *Certified Programs and Proofs*. Chris Hawblitzel and Dale Miller, editors. Springer Berlin Heidelberg, Berlin, Heidelberg, 92–108. ISBN: 978-3-642-35308-6. doi:10.1007/978-3-642-35308-6_10.

[12] Harsh Raju Chamarthi, Peter C. Dillinger, Matt Kaufmann, and Panagiotis Manolios. 2011. Integrating Testing and Interactive Theorem Proving. In *Proceedings of the 10th International Workshop on the ACL2 Theorem Prover* (Electronic Proceedings in Theoretical Computer Science). David Hardin and Julien Schmaltz, editors. Vol. 70. Open Publishing Association, Austin, Texas, USA, 4–19. doi:10.4204/EPTCS.70.1.

[13] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*, 19 pages. https://openreview.net/forum?id=ktrw68Cmu9c.

[14] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. (2021). https://arxiv.org/abs/2107.03374 arXiv: 2107.03374 [cs.LG].

[15] Aaron Councilman, David Jiahao Fu, Aryan Gupta, Chengxiao Wang, David Grove, Yu-Xiong Wang, and Vikram Adve. 2025. Towards Formal Verification of LLM-Generated Code from Natural Language Prompts. In Submission. (2025). https://arxiv.org/abs/2507.13290 arXiv: 2507.13290 [cs.PL].

[16] Benjamin Darnell, Hetarth Chopra, Aaron Councilman, David Grove, Yu-Xiong Wang, and Vikram Adve. 2024. An Empirical Comparison of Code Generation Approaches for Ansible. In *Proceedings of the ACM/IEEE 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering* (InteNSE '24). Association for Computing Machinery, Lisbon, Portugal, 1–6. ISBN: 9798400705649. doi:10.1145/3643661.3643951.

[17] Edsger W. Dijkstra. 1979. On the foolishness of "natural language programming". In *Program Construction: International Summer School*. Friedrich L. Bauer et al., editors. Springer Berlin Heidelberg, Berlin, Heidelberg, 51–53. ISBN: 978-3-540-35312-6. doi:10.1007/BFb0014656.

[18] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. 2025. A Survey on Code Generation with LLM-based Agents. (2025). https://arxiv.org/abs/2508.00083 arXiv: 2508.00083 [cs.SE].

[19] Shihan Dou et al. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. (2024). https://arxiv.org/abs/2407.06153 arXiv: 2407.06153 [cs.SE].

[20] Carl Eastlund. 2009. DoubleCheck your theorems. In *Proceedings of the Eighth International Workshop on the ACL2 Theorem Prover and Its Applications* (ACL2 '09). Association for Computing Machinery, Boston, Massachusetts, USA, 42–46. ISBN: 9781605587424. doi:10.1145/1637837.1637844.

[21] Sigrid Eldh, Hans Hansson, and Sasikumar Punnekkat. 2011. Analysis of Mistakes as a Method to Improve Test Case Design. In *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*, 70–79. doi:10.1109/ICST.2011.52.

[22] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? *Proc. ACM Softw. Eng.*, 1, FSE, Article 84, (July 2024), 24 pages. doi:10.1145/3660791.

[23] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (ESEC/FSE 2023). Association for Computing Machinery, San Francisco, CA, USA, 1229–1241. ISBN: 9798400703270. doi:10.1145/3611643.3616243.

[24] Nobert E. Fuchs, Uta Schwertel, and Sunna Torge. 1999. Controlled Natural Language Can Replace First-Order Logic. In *14th IEEE International Conference on Automated Software Engineering*, 295–298. doi:10.1109/ASE.1999.802325.

[25] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. (2024). https://arxiv.org/abs/2407.21783 arXiv: 2407.21783 [cs.AI].

[26] Michael Greenberg and Austin J. Blatt. 2019. Executable formal semantics for the POSIX shell. *Proc. ACM Program. Lang.*, 4, POPL, Article 43, (Dec. 2019), 30 pages. doi:10.1145/3371111.

[27] Xiaodong Gu, Meng Chen, Yalan Lin, Yuhan Hu, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, and Juhong Wang. 2025. On the Effectiveness of Large Language Models in Domain-Specific Code Generation. *ACM Transactions on Software Engineering and Methodology*, 34, 3, (Feb. 2025), 1–22. doi:10.1145/3697012.

[28] Sumit Gulwani. 2010. Dimensions in program synthesis. In *Proceedings of the 12th International ACM SIGPLAN Symposium on Principles and Practice of Declarative Programming* (PPDP '10). Association for Computing Machinery, Hagenberg, Austria, 13–24. ISBN: 9781450301329. doi:10.1145/1836089.1836091.

[29] Daya Guo et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. (2024). https://arxiv.org/abs/2401.14196 arXiv: 2401.14196 [cs.SE].

[30] Catalina Hallett, Donia Scott, and Richard Power. 2007. Composing Questions through Conceptual Authoring. *Computational Linguistics*, 33, 1, (Mar. 2007), 105–133. doi:10.1162/coli.2007.33.1.105.

[31] Binyuan Hui et al. 2024. Qwen2.5-Coder Technical Report. (2024). https://arxiv.org/abs/2409.12186 arXiv: 2409.12186 [cs.CL].

[32] Piyush Jha, Prithwish Jana, Pranavkrishna Suresh, Arnav Arora, and Vijay Ganesh. 2025. RLSF: Fine-tuning LLMs via Symbolic Feedback. (2025). https://arxiv.org/abs/2405.16661 arXiv: 2405.16661 [cs.CL].

[33] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*, 51 pages. https://openreview.net/forum?id=VTF8yNQM66.

[34] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. 2024. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. (2024). https://arxiv.org/abs/2410.03981 arXiv: 2410.03981 [cs.SE].

[35] Gerwin Klein et al. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (SOSP '09). Association for Computing Machinery, Big Sky, Montana, USA, 207–220. ISBN: 9781605587523. doi:10.1145/1629575.1629596.

[36] Tobias Kuhn. 2014. A Survey and Classification of Controlled Natural Languages. *Computational Linguistics*, 40, 1, (Mar. 2014), 121–170. doi:10.1162/COLI_a_00168.

[37] Leonidas Lampropoulos. 2018. *Random Testing for Language Design*. PhD thesis. University of Pennsylvania. ISBN: 978-0-438-42440-1. https://www.proquest.com/dissertations-theses/random-testing-language-design/docview/2118029142/se-2.

[38] Xavier Leroy. 2006. Formal Certification of a Compiler Back-end or: Programming a Compiler with a Proof Assistant. In *Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (POPL '06). Association for Computing Machinery, Charleston, South Carolina, USA, 42–54. ISBN: 1595930272. doi:10.1145/1111037.1111042.

[39] Junjie Li, Aseem Sangalay, Cheng Cheng, Yuan Tian, and Jinqiu Yang. 2024. Fine Tuning Large Language Model for Secure Code Generation. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering* (FORGE '24). Association for Computing Machinery, Lisbon, Portugal, 86–90. ISBN: 9798400706097. doi:10.1145/3650105.3652299.

[40] Raymond Li et al. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research*, 43 pages. https://openreview.net/forum?id=KoFOg41haE.

[41] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (ICSE '24) Article 52. Association for Computing Machinery, Lisbon, Portugal, 13 pages. ISBN: 9798400702174. doi:10.1145/3597503.3608128.

[42] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*. A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors. Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 21558–21572. https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf.

[43] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. 2023. "What It Wants Me To Say": Bridging the Abstraction Gap Between End-User Programmers and Code-Generating Large Language Models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (CHI '23) Article 598. Association for Computing Machinery, Hamburg, Germany, 31 pages. ISBN: 9781450394215. doi:10.1145/3544548.3580817.

[44] Anton Lozhkov et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. (2024). https://arxiv.org/abs/2402.19173 arXiv: 2402.19173 [cs.SE].

[45] Mayank Mishra et al. 2024. Granite Code Models: A Family of Open Foundation Models for Code Intelligence. (2024). https://arxiv.org/abs/2405.04324 arXiv: 2405.04324 [cs.AI].

[46] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-Constrained Code Generation with Language Models. *Proc. ACM Program. Lang.*, 9, PLDI, Article 171, (June 2025), 26 pages. doi:10.1145/3729274.

[47] Dung Manh Nguyen, Thang Chau Phan, Nam Le Hai, Tien-Thong Doan, Nam V. Nguyen, Quang Pham, and Nghi D. Q. Bui. 2025. CodeMMLU: A Multi-Task Benchmark for Assessing Code Understanding & Reasoning Capabilities of CodeLLMs. In *The Thirteenth International Conference on Learning Representations*, 59 pages. https://openreview.net/forum?id=CahIEKCu5Q.

[48] OpenAI et al. 2024. GPT-4 Technical Report. (2024). https://arxiv.org/abs/2303.08774 arXiv: 2303.08774 [cs.CL].

[49] Hila Peleg, Shachar Itzhaky, and Sharon Shoham. 2018. Abstraction-Based Interaction Model for Synthesis. In *Verification, Model Checking, and Abstract Interpretation*. Isil Dillig and Jens Palsberg, editors. Springer International Publishing, Cham, 382–405. ISBN: 978-3-319-73721-8. doi:10.1007/978-3-319-73721-8_18.

[50] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (CCS '23). Association for Computing Machinery, Copenhagen, Denmark, 2785–2799. ISBN: 9798400700507. doi:10.1145/3576915.3623157.

[51] Saurabh Pujar et al. 2023. Automated Code Generation for Information Technology Tasks in YAML through Large Language Models. (2023). https://arxiv.org/abs/2305.02783 arXiv: 2305.02783 [cs.SE].

[52] Red Hat. 2025. Red Hat Ansible Lightspeed. (2025). https://www.redhat.com/en/technologies/management/ansible/ansible-lightspeed.

[53] Red Hat. 2025. What is Ansible? (2025). https://www.ansible.com.

[54] Red Hat, Inc. 2025. command-instead-of-module, Ansible Lint Documentation. (2025). https://ansible.readthedocs.io/projects/lint/rules/command-instead-of-module/#correct-code.

[55] Grigore Roșu and Traian Florin Șerbănuță. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming*, 79, 6, 397–434. doi:10.1016/j.jlap.2010.03.012.

[56] Baptiste Rozière et al. 2024. Code Llama: Open Foundation Models for Code. (2024). https://arxiv.org/abs/2308.12950 arXiv: 2308.12950 [cs.CL].

[57] Priyam Sahoo, Saurabh Pujar, Ganesh Nalawade, Richard Genhardt, Louis Mandel, and Luca Buratti. 2024. Ansible Lightspeed: A Code Generation Service for IT Automation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (ASE '24). Association for Computing Machinery, Sacramento, CA, USA, 2148–2158. ISBN: 9798400712487. doi:10.1145/3691620.3695277.

[58] Rolf Schwitter. 2002. English as a Formal Specification Language. In *Proceedings. 13th International Workshop on Database and Expert Systems Applications*, 228–232. doi:10.1109/DEXA.2002.1045903.

[59] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: An empirical study. *Empirical Software Engineering*, 30, (Feb. 2025). doi:10.1007/s10664-025-10614-4.

[60] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2025. SynCode: LLM Generation with Grammar Augmentation. *Transactions on Machine Learning Research*, 48 pages. https://openreview.net/forum?id=HiUZtgAPoH.

[61] Jianxun Wang and Yixiang Chen. 2023. A Review on Code Generation with LLMs: Application and Evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, 284–289. doi:10.1109/MedAI59581.2023.00044.

[62] Yuxiang Wei et al. 2024. SelfCodeAlign: Self-Alignment for Code Generation. In *Advances in Neural Information Processing Systems*. A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors. Vol. 37. Curran Associates, Inc., 62787–62874. https://proceedings.neurips.cc/paper_files/paper/2024/file/72da102da91a8042a0b2aa968429a9f9-Paper-Conference.pdf.

[63] Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems*. S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors. Vol. 35. Curran Associates, Inc., 32353–32368. https://proceedings.neurips.cc/paper_files/paper/2022/file/d0c6bc641a56bebee9d985b937307367-Paper-Conference.pdf.

[64] Yutong Wu et al. 2025. InverseCoder: Self-improving Instruction-Tuned Code LLMs with Inverse-Instruct. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39, 24, (Apr. 2025), 25525–25533. doi:10.1609/aaai.v39i24.34742.

[65] Daniel M. Yellin. 2023. The Premature Obituary of Programming. *Communications of the ACM*, 66, 2, (Jan. 2023), 41–44. doi:10.1145/3555367.

[66] Chuheng Zhang, Wei Shen, Li Zhao, Xuyun Zhang, Xiaolong Xu, Wanchun Dou, and Jiang Bian. 2025. Policy Filtration for RLHF to Mitigate Noise in Reward Models. (2025). https://arxiv.org/abs/2409.06957 arXiv: 2409.06957 [cs.LG].

[67] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.*, 2, ISSTA, Article ISSTA022, (June 2025), 23 pages. doi:10.1145/3728894.

[68] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. 2022. Fuzzing: A Survey for Roadmap. *ACM Comput. Surv.*, 54, 11s, Article 230, (Sept. 2022), 36 pages. doi:10.1145/3512345.

[69] Terry Yue Zhuo et al. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *The Thirteenth International Conference on Learning Representations*, 55 pages. https://openreview.net/forum?id=YrycTjllL0.

[70] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (MAPS 2022). Association for Computing Machinery, San Diego, CA, USA, 21–29. ISBN: 9781450392730. doi:10.1145/3520312.3534864.